Expert Systems With Applications 318 (2026) 131805

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

State recommender system for actuator devices in smart homes: Integration of deep reinforcement learning and implicit feedback

Denis Robson Dantas Boaventura [a,*], Eduardo Ferreira da Silva [a], Mayki dos Santos Oliveira [a], Diego Corrêa da Silva [b], Frederico Araújo Durão [a]

[a] Federal University of Bahia, Salvador, Bahia, Brazil
[b] University of Klagenfurt, Carinthia, Klagenfurt, Austria

ARTICLE INFO

Keywords:
Smart homes
Recommender systems
Reinforcement learning
Multi-agent recommender system
Smart environment
Smart device

ABSTRACT

The rapid expansion of the Internet of Things (IoT) has provided the technological foundation for smart homes, in which interconnected devices enable the environment to adapt to residents' needs. Yet, this proliferation of IoT-enabled devices also introduces a significant challenge: managing and coordinating their multiple operational states, which extend far beyond a simple "switch on/switch off". Aiming to automate the action process and anticipate user actions, this study proposes a state recommender system for actuator devices in smart homes, developed using deep reinforcement learning algorithms integrated with implicit feedback. This approach enables composite state encoding and coordination among multiple agents, thereby anticipating user needs and adapting to dynamic routines. In addition to using a dataset captured from a real-world IoT environment, we use a smart home simulator to generate two datasets based on three different routines and perform experiments with two reinforcement learning algorithms, Deep Q-Learning and Differential Semi-gradient n-step SARSA, benchmarking their performance against an online Supervised Learning baseline based on Behavior Cloning. Additionally, we tested these algorithms using two-state approaches: simple and composite. The results confirm the system's successful adaptation to user routines across both approaches, underscoring its potential to enhance personalization in smart home environments.

1. Introduction

Smart Home (SH) represents a key application domain of the Internet of Things (IoT), where interconnected devices collaborate to enhance the quality of life through automation, efficiency, and personalization. The IoT, broadly defined as a network of physical objects embedded with sensors, software, and connectivity, enables these devices to collect and exchange data, creating responsive and intelligent environments (Rashdan et al., 2025). Within the SH context, IoT technologies facilitate seamless integration among lighting, climate control, security systems, appliances, and user interfaces, fostering environments that are adaptive to their inhabitants' habits and preferences. This convergence of connectivity and automation not only optimizes energy usage and convenience but also introduces new challenges related to system complexity, privacy, and interoperability (Yang et al., 2017).

According to Ashton (1999), the interconnection of (IoT) has driven major advances in SHs. Evolving from early device automation toward increasingly autonomous environments that must manage sophisticated devices and complex interactions while continuously adapting to residents' preferences (Kim et al., 2020; Yang et al., 2017). With the incorporation of artificial intelligence, SHs can operate more independently, but this autonomy raises challenges in balancing predictability and user control to avoid compromising users (Davidoff et al., 2006). In this context, Recommender Systems (RSs) have emerged as a promising approach for leveraging IoT-generated data to deliver personalized suggestions and enhance the user experience, making them a key research topic in smart environments (Ricci et al., 2022). However, this intersection remains immature due to the lack of standardized methodologies and public datasets, as well as the limited availability of source code, which hinders reproducibility and slows progress in the smart-home field (Afsar et al., 2022; Himeur et al., 2021).

Conventional RSs, which rely on historical data and pre-established behavior patterns, face limitations when applied to dynamic scenarios, such as SHs. They struggle to adapt quickly to changes in users' preferences, compromising continuous personalization (Zhang et al., 2022). Additionally, modeling preferences in real time is challenging, as many

* Corresponding author.
E-mail addresses: denis.boaventura@ufba.br (D.R.D. Boaventura), eduardoferreira@ufba.br (E.F. da Silva), maykioliveira@ufba.br (M. dosS. Oliveira), diego.correa@aau.at (D.C. da Silva), fdurao@ufba.br (F.A. Durão).

https://doi.org/10.1016/j.eswa.2026.131805
Received 26 September 2025; Received in revised form 10 February 2026; Accepted 20 February 2026
Available online 10 March 2026
0957-4174/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

systems fail to capture preferences robustly, especially when they rely solely on explicit feedback, which has limitations such as users' reluctance to provide it and its susceptibility to momentary influences (Ricci et al., 2022). Within this gap, integrating Reinforcement Learning (RL) into RS for SHs emerges as a promising approach to address these challenges. This integration enables RSs to operate in real time, leveraging both implicit and explicit feedback, and aligning with users' expectations for convenience, comfort, and continuous personalization (Sutton & Barto, 2018). The application of RL not only fosters more responsive interactions between devices and residents, but also meets the demand for dynamic systems that continuously adapt to changes in users' preferences and behaviors, thereby promoting a smoother, more efficient experience within SHs.

This work [1] proposes an event-driven RS, based on the integration of Deep Reinforcement Learning (DRL) and implicit feedback to orchestrate actuator devices in SHs. The research addresses an underdeveloped topic with many gaps. The contributions of this work to the state of the art are the following:

- A recommendation system for smart homes that models implicit feedback and its corresponding reward function to enable autonomous personalization for smart devices.
- An analysis of the adaptability of an on-policy and an off-policy reinforcement learning algorithm applied to the smart home domain.
- Two domain-specific action selection strategies adapted for reinforcement learning systems in environments with the possibility of routine change.
- A new metric based on the Hamming Score for evaluating predictions for smart devices with a composite state.

Our approach has the potential to overcome the limitations of traditional recommendation techniques by effectively responding to changing resident preferences and providing a continuously personalized experience. This paper is structured as follows: Section 2 presents related work; Section 3 discusses the concepts underlying the main proposal; Section 4 details our solution; and Section 5 describes the dataset and experimental methodology. Finally, Sections 6 and 7 present a discussion of the results and the findings.

2. Related works

This section synthesizes prior research on smart homes and RS, focusing on the differences in objectives, learning, and assumptions among studies. We organize the literature into three thematic groups: energy efficiency and consumption management; activity modeling, context-aware automation, and multi-label smart-home prediction; and reinforcement learning for recommender systems. Across these works, we emphasize recurring dimensions such as optimization goals (energy, security, comfort), feedback modeling (explicit vs. implicit), decision cadence (periodic vs. event-driven), orchestration scope (single subsystem vs. whole-home), and device-state complexity (binary vs. composite).

2.1. Energy efficiency and consumption management

Most recent studies on SHs prioritize energy efficiency and consumption management, frequently leveraging RSs to support decision-making (Alomar et al., 2022; Asare-Bediako et al., 2013; B et al., 2024; Jiménez-Bravo et al., 2019; Kaur et al., 2019; Khezzane et al., 2024; Luo et al., 2021; Rodríguez Fernández et al., 2015). These works differ in how they collect and exploit contextual and usage signals: some emphasize real-time monitoring and responsive control (Alomar et al., 2022), whereas others focus on load and price forecasting to guide energy-aware decisions (Jiménez-Bravo et al., 2019). Methodologically, approaches range from recommendation techniques such as Collaborative Filtering (Luo et al., 2021) to Multi-agent System (MAS)-based architectures that coordinate devices and policies in distributed settings (Asare-Bediako et al., 2013; Kaur et al., 2019). Recent advances also incorporate multi-layer, multi-objective optimization to jointly minimize the consumption cost and the peak-to-average ratio while maximizing user satisfaction (Khezzane et al., 2024). Beyond energy management, security-oriented mechanisms, such as user-based fingerprint systems, can indirectly improve energy efficiency by enabling automated appliance control with differentiated access levels (B et al., 2024).

In a broader scope, Rashdan et al. (2025) integrates remote monitoring, energy management, and user comfort through customizable interfaces provided by the *Blynk* mobile application. Among energy-focused approaches, Asare-Bediako et al. (2013) is notable for incorporating feedback; however, their feedback is delivered *to* the user to increase consumption awareness. In contrast, our work models user actions as *implicit feedback* that directly drives the learning process of the proposed algorithms. Overall, these studies provide foundational architectures for deploying RSs in smart environments, informing the system design choices in our proposal.

2.2. Activity modeling, context-aware automation, and multi-label smart-home prediction

Related research explores the optimization of automation and interaction to enhance convenience and precision in SHs (Cook et al., 2003; Jeon et al., 2022). While Jeon et al. (2022) proposes a RS based on context-aware encoders that cluster user actions using temporal and sequential context, Cook et al. (2003) predicts activities using an ensemble strategy that combines multiple predictors via voting. Complementarily, da Silva et al. (2025) investigates multi-label learning for SHs, emphasizing the challenge of predicting interdependent device states and proposing a data transformation methodology aligned with real IoT structures. However, unlike our event-driven setting, their model is trained on second-by-second data and performs predictions at fixed time intervals, which changes the operational assumptions about when decisions are triggered and updated. Moreover, although (Jeon et al., 2022) is closer to our setting as a SH state RS, it does not incorporate user feedback as a learning input. In addition, some approaches (e.g., Cook et al., 2003; Sun et al., 2025) pursue objectives that diverge from our primary focus on event-driven personalization and whole-home actuator orchestration.

2.3. Reinforcement learning for recommender systems

Recently, data-driven approaches, including Machine Learning (ML) techniques, have transcended their traditional boundaries, demonstrating remarkable efficacy in solving complex optimization problems across a wide spectrum of engineering disciplines. Recent studies have highlighted the robust adaptability of these learning-based methods in fields ranging from multidomain nanomaterial design (Wang et al., 2024) and chemical engineering applications in nanomaterials (Yang et al., 2024) to the accelerated discovery and optimization of metal-organic frameworks (Wang et al., 2025). Following this trend of successful deployment in complex dynamic systems, RL has also emerged as a promising approach for orchestrating SH environments.

In this context, user–system interaction is inherently sequential, which allows recommendation to be framed as a Markov Decision Process (MDP) and optimized via RL (Afsar et al., 2022). This formulation is particularly suitable for dynamic environments, where RSs must continuously revise preference estimates and respond to rapidly changing routines (Chen et al., 2023). In smart-home contexts, several studies employ RL to optimize device behavior and improve RSs (CHÉ et al., 2010; Gokul et al., 2016; Mirra et al., 2018; Mozer, 1998; Reaz et al., 2008; Yu et al., 2020). Their targets range from balancing comfort and energy savings (including image-based sensing) (Mozer, 1998) to energy

[1] Code and datasets for this work: https://github.com/denisbvnt/Recommender-System-for-Smart-Homes-Based-on-Reinforcement-Learning

management for Heating, Ventilation, and Air Conditioning (HVAC) and Energy Storage System (ESS) systems (Yu et al., 2020) to the automation of lights and appliances (Mirra et al., 2018). Other works learn user interaction patterns (CHÉ et al., 2010), exploit image processing to anticipate actions (Gokul et al., 2016), or adopt MAS frameworks to support interaction prediction and accessibility-oriented automation (Reaz et al., 2008).

A key differentiator across these RL-based systems is how rewards and feedback are modeled. For example, Mozer (1998) updates policies conditional on non-negative user feedback while encouraging energy-efficient transitions; Yu et al. (2020) penalizes energy consumption and temperature deviation; Mirra et al. (2018) employs binary rewards; Gokul et al. (2016) derives rewards from visual environment cues; and Reaz et al. (2008) defines rewards from user feedback. In contrast, our proposal utilizes DRL to anticipate users' future actions and orchestrate all home actuators through an event-driven decision-making process. Our model explicitly accounts for the complexity of real-world smart devices with composite states (e.g., switch, temperature, color, intensity), treats device interactions as implicit feedback, and computes rewards for each device-state recommendation to support continual personalization under routine changes.

3. Background

3.1. Recommender systems

We can define RSs as software tools and techniques that provide suggestions of items likely to be of interest to a particular user (Ricci et al., 2022). These suggestions are made by predicting which product or service will most please users, based on their preference history. In a RS approach, "item" is a generic term denoting what the system suggests to a user, "user" is a generic term for whom the system provides recommended items, and "interactions" are records of users' actions and behaviors toward items, including purchases, browsing, clicks, ratings, along with contextual data (such as time and location) (Ricci et al., 2022).

During user sessions with the system, interactions are recorded and serve as the database for user modeling. According to Ricci et al. (2022), two main techniques can be adopted to collect user data on the items they interact with: in most interactions, users may either interact positively or negatively with items. In this context, implicit feedback is particularly valuable in SH environments. Users rarely provide explicit ratings for system actions. Instead, their preferences are reflected in operational patterns, such as adjusting thermostat settings, turning lights "on" or "off", or changing appliance modes.

3.2. Deep reinforcement learning algorithms

According to Sutton and Barto (2018), RL concerns learning a policy that maps situations to actions to maximize a numerical reward over time through interaction with an environment. DRL extends this paradigm by using deep neural networks to approximate value functions and/or policies, enabling learning in complex, high-dimensional settings with large state and action spaces. A SH environment fits these characteristics, featuring a variety of smart devices with states comprising multiple attributes. There are numerous DRL algorithms, each better suited for solving specific types of problems. We will use the Deep Q-Learning (DQL) and Differential Semi-gradient n-step SARSA (DSGnSARSA) algorithms, as they possess distinct characteristics that enable a comparison between on-policy and off-policy methods, providing a comprehensive performance analysis across different contexts.

In this work, for both DQL and DSGnSARSA, we use the same neural network architecture as a function approximator. The network employs an input layer that receives the *Environment state* S, followed by three hidden layers (64-128-64 neurons) with Rectified Linear Unit (ReLU) activation function. The output layer is dynamic and represents the actions the agent can take, expanding as the action space grows.

3.2.1. Deep Q-learning

DQL is an off-policy algorithm that uses neural networks as a non-linear function approximator in Q-Learning (Mnih et al., 2013). This algorithm employs two distinct policies: a *behavior policy* for exploration and a *target policy* for updates, both implemented as neural networks. The method addresses two fundamental challenges: 1) temporal correlations via *replay buffer* and 2) target instability with *target networks*.

The Q-value update function of the Q-Learning algorithm is a sample-based version of value iteration that iteratively applies the Bellman optimization equation. It performs updates based on the difference between estimated Q-values and target Q-values.

3.2.2. Differential semi-gradient n-step SARSA

This algorithm is a variant of the SARSA algorithm that combines three essential components for RL: 1) *Differential RL*, which replaces the discount factor γ with a focus on the average reward per time step $\bar{r}$; 2) *n-step bootstrapping*, which considers an n-step window to balance bias and variance; and 3) *semi-gradient*, which uses gradients for function approximation optimization while ignoring the target's dependency on current parameters (Sutton & Barto, 2018).

While in DQL we seek the maximum over next-action values (thus requiring the agent to learn which action is better than another), in Differential Semi-Gradient n-Step SARSA, we use the estimated next-action value itself as the target for Q-value updates. This target will change whenever the agent takes an exploratory action. Like SARSA, it is an on-policy algorithm because the agent initializes the value of the next action to be taken by sampling from its behavior policy. Moreover, unlike DQL, its value function estimate is updated after every transition in the environment.

3.3. Action selection techniques

One of the most significant dilemmas in RL is the trade-off between exploration and exploitation (Morales, 2020; Sutton & Barto, 2018). In exploration, the agent's role is to discover new actions to gather more information about the environment. In exploitation, the agent leverages known information to maximize reward, meaning it will always tend to select the action with the highest value. Properly balancing these two behavioral modes is the primary objective of action selection techniques (Morales, 2020; Sutton & Barto, 2018). Action selection techniques in RL determine which action an agent takes in each state, typically by balancing exploration of new actions with exploitation of known high-reward actions to maximize long-term return.

Choosing an appropriate action selection mechanism significantly impacts both learning and agent performance, since in RL, agents learn from their own acquired experiences (whether from actions deemed good or bad) and act based on what has been learned so far. Morales (2020) (Morales, 2020) discusses some of these action selection techniques; below we describe those most relevant to this research:

- *ϵ-greedy*: A technique that combines exploration and exploitation, where ϵ is the probability that the agent selects a random action. It consists of acting most of the time greedily and exploring randomly occasionally,

$$a_t = \begin{cases} \arg\max Q_t(s,a), & \text{with probability } 1-\epsilon_t \\ \text{any action}, & \text{with probability } \epsilon_t \end{cases}. \tag{1}$$

- **Decaying ϵ-greedy**: An adaptation of ϵ-greedy where ϵ_t decreases over time. It starts with a high exploration rate and gradually reduces it, favoring exploitation as learning progresses. The exploration rate

ϵ_t at time step t is defined as:

$$\epsilon_t = \max(\epsilon_{\min}, \epsilon_0 \cdot \lambda^t),$$

$$a_t = \begin{cases} \arg\max_{a \in \mathcal{A}} Q_t(s,a), & \text{with probability } 1-\epsilon_t \\ \text{uniform random action}, & \text{with probability } \epsilon_t \end{cases}, \quad (2)$$

where ϵ_0 is the initial exploration rate ($\epsilon_0 \in [0,1]$), λ is the decay rate ($\lambda \in (0,1)$), $\epsilon_{\min}$ is the minimum exploration threshold, and a_t is the selected action.

- **Softmax**: Samples actions from a probability distribution over action-value estimates, where selection probability is proportional to each action's current estimated value. The probability $\pi_t(a|s)$ of selecting action a in state s is determined by the Boltzmann distribution:

$$\pi_t(a|s) = \frac{e^{Q_t(s,a)/\tau}}{\sum_{b \in \mathcal{A}} e^{Q_t(s,b)/\tau}}, \quad (3)$$

where $\tau \in \mathbb{R}^+$ is the temperature parameter controlling exploration intensity. Lower τ values increase the probability of selecting the highest-valued action, tending towards a greedy policy as $\tau \to 0$.

In this work, the softmax and decaying ϵ-greedy algorithms were chosen not only for experimentation in our proposed RS, but also as foundational baselines for two domain-specific exploration strategies. This choice is justified by the need to balance exploration and exploitation in dynamic environments, as well as the differences between these two algorithms. While the softmax method offers continuous exploration proportional to the value of the actions, which is ideal for policy refinement, the decaying ϵ-greedy adopts an initial strategy of intense exploration that is gradually abandoned in favor of optimization. This is essential in scenarios where the optimal system behavior emerges over time. For the decaying ϵ-greedy exploration strategy, the parameters were configured with a *maximum* ϵ of 1, which decayed to a *minimum* ϵ of 0. The ϵ *decay rate* was set to 0.9 and was applied with a *decay frequency* of every 10 timesteps. For the softmax exploration strategy, the *temperature* (τ) was set to 100.

4. State recommender system for actuator devices in smart homes

This work proposes an event-driven RS for SHs that recommends actuator device states using multiple DRL agents. Through simulated experiments, we assess whether implicit user feedback can optimize each device-specific controller policy. Since implicit feedback infers intent from observed behavior, the reward and feedback modeling must be carefully defined to avoid erroneous inferences that degrade system performance. In our framework, the RS is the delivered application, DRL provides the learning and interaction mechanism, and the SH domain supplies the simulation and validation environment.

The RS logic is triggered by events such as user actions, device messages, or changes in system state. These events trigger the execution of specific functions and methods to process and respond to them in real-time. The proposed architecture (Fig. 1) incorporates the environment and features multiple collaborative agents that orchestrate smart actuator devices through state recommendations.

The workflow, as illustrated in Fig. 1, is initiated when new events generated by the inhabitants' direct actions on the environment are detected. This ensures that agents always receive the home's current state in an up-to-date manner. Including not only the state of each device but also a temporal representation of when the changes were made. The following steps outline the sequential flow of the proposed recommender system (Fig. 1) and highlight how events are processed and how agents interact to generate and refine device recommendations within the smart home environment.

1. The user's interaction with a smart device modifies the environment's state, thus creating a new event;
2. The RS receives the house model at the current moment (timestamps and device states);
3. The *Data Processing Agent* processes, treats, and retransmits the data, transforming the received house model into environment state S;
4. Each of the *Recommender agents* generates state recommendations for the actuator device to which they refer, based on their trained DRL models;
5. In the next event, the user's implicit feedback is inferred from their behavior and used to calculate rewards that will be sent to the *Recommender agents* and later used in training the DRL models.

The environment is an abstraction of the RL framework that represents the world in which the RS interacts. Our SH environment comprises two types of devices:

- **Sensor devices:** They are responsible for collecting internal or external data, detecting information about the *Environment* and the inhabitants of the home, and monitoring variables such as motion, temperature, humidity, and air quality. They are used exclusively to observe the *Environment* and identify the behavior of the inhabitants.
- **Actuator devices:** They receive action signals from the Controller Agent or directly from the inhabitants, transforming the signals into physical changes in the *Environment* and continuously indicating their state. Examples include lights, electronic locks, smart plugs, and air conditioners.

The state recommender system employs multiple autonomous DRL agents that interact collaboratively to achieve both common and individual goals. The need to assign an agent to each actuator device arises from the inherent complexity of these devices and, by extension, the environment in which they operate. Agents operate through a cycle of perception, action, reward reception, and learning, enabling the optimization of adaptive strategies.

- **Data processing agent:** It is responsible for processing the data received from the *Environment* and sending it to the *Recommender agents*, ensuring quality for learning. Its tasks include encoding non-numeric values, creating relevant features, and extracting temporal attributes. Crucially, to handle the high dimensionality of composite states (combinations of different device attributes), we employ a *Dynamic Integer Encoding* strategy. As illustrated in Fig. 2, each unique message tuple (representing a specific device configuration) represents a key in a dynamic dictionary. When a new, unique configuration is observed, it is assigned a new, incremental scalar integer starting from 0. If the configuration has appeared in the history, it is mapped to its existing integer code. This ensures that the neural networks' input dimensionality remains compact (scalar) regardless of the device's complexity.
- **Recommender agents:** It is responsible for recommending states for actuator devices under their control, acting directly on the Environment. Each agent is associated with a single actuator device and employs a DRL approach, observing states, applying actions, receiving rewards, and processing transitions. Internally, agents follow steps such as action selection, state transition, transition storage, and the generation of specific recommendations for their actuator devices. The learning process is performed every n timesteps using a set of accumulated transitions.

Table 1 presents the mathematical notation adopted in the proposed solution. It summarizes the key symbols used to represent the environment state, user preferences, agent actions, and reward structure within the smart home context. For each actuator, the table distinguishes between primary and secondary states for both user-defined configurations and agent-generated recommendations. Additionally, it includes the reward components associated with these states, which guide the learning process of the DRL agents.

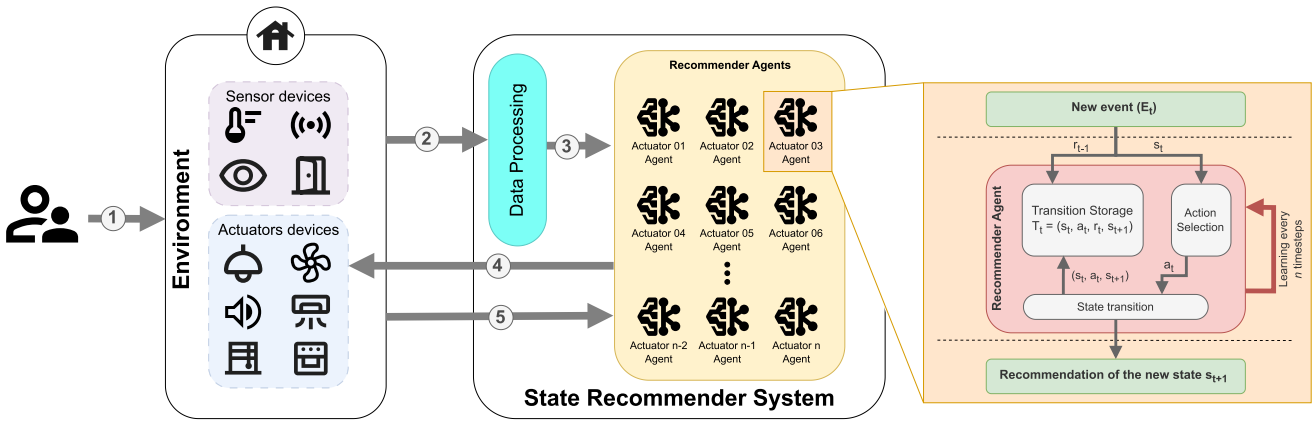

Fig. 1. Proposed recommender system architecture for smart homes.

timestamp	OFFICE AIR CONDITIONER	OFFICE LIGHT
2024-7-12 09:15:01	{'status': [{'code': 'switch', 'value': 'ON'}, {'code': 'temp_set', 'value': 24}, {'code': 'mode', 'value': 'WIND'}]}	{'status': [{'code': 'switch', 'value': 'OFF'}, {'code': 'sound_volume', 'value': 40}]}
2024-7-12 09:23:13	{'status': [{'code': 'switch', 'value': 'ON'}, {'code': 'temp_set', 'value': 27}, {'code': 'mode', 'value': 'AUTO'}]}	{'status': [{'code': 'switch', 'value': 'OFF'}, {'code': 'sound_volume', 'value': 40}]}
2024-7-12 10:07:25	{'status': [{'code': 'switch', 'value': 'OFF'}, {'code': 'temp_set', 'value': 27}, {'code': 'mode', 'value': 'AUTO'}]}	{'status': [{'code': 'switch', 'value': 'ON'}, {'code': 'sound_volume', 'value': 40}]}
2024-7-12 10:32:41	{'status': [{'code': 'switch', 'value': 'ON'}, {'code': 'temp_set', 'value': 27}, {'code': 'mode', 'value': 'AUTO'}]}	{'status': [{'code': 'switch', 'value': 'OFF'}, {'code': 'sound_volume', 'value': 40}]}

timestamp	OFFICE AIR CONDITIONER	OFFICE LIGHT
2024-7-12 09:15:01	0	0
2024-7-12 09:23:13	1	0
2024-7-12 10:07:25	2	1
2024-7-12 10:32:41	1	0

Fig. 2. Dynamic integer encoding of device states into scalar values.

Table 1
Notation of the proposed solution.

Symbol	Description
S	Full state of the environment
S^S	Set of states of all sensor devices
S^A	Set of states of all actuator devices
S^T	Set of states of all temporal variables
S_i^S	Complete state of sensor i
S_i^A	Complete state of actuator i
$S_{i,p}^A$	Primary state of actuator i
S_{i,s_j}^A	Secondary state j of actuator i
$S_{i,p}^S$	Primary state of sensor i
S_{i,s_j}^S	Secondary state j of sensor i
A_i	Full state recommended by the agent for actuator i
$A_{i,p}$	Primary state recommended by the agent for actuator i
A_{i,s_j}	Secondary state j recommended by the agent for actuator i
U_i	Full state chosen by the user for actuator i
$U_{i,p}$	Primary state chosen by the user for actuator i
U_{i,s_j}	Secondary state j chosen by the user for actuator i
R_i	Total reward for the recommendation made by the agent for actuator i
$R_{i,p}$	Reward for the primary state of actuator i, where $R_{i,p} \in \{0, 1\}$
$R_{i,s}$	Reward for the set of secondary states of actuator i
R_{i,s_j}	Reward for secondary state j of actuator i, where $R_{i,s_j} \in [0, 1]$
n_i	Total number of states (primary + secondary) for actuator i
$1(\cdot)$	Indicator function that equals 1 if the condition is true, and 0 otherwise
WD	Set of values representing the weekday
HI	Set of values representing the hour-level intervals
MI	Set of values representing the minute-level interval

4.1. User interaction with the environment and implicit feedback

In a SH, the users represent the house's inhabitants. To improve user satisfaction, RS requires mechanisms to transform user interactions into learning opportunities. Our approach automatically models implicit feedback by leveraging home data, removing the dependency on direct user input (explicit feedback). Fig. 3 demonstrates this modeling for a single actuator device. The same formulation is applied uniformly to all actuator devices in the *Environment*.

At each event, the RS generates recommendations for the actuator devices' states and stores them in the environment. When the next event occurs, and a user interacts with any device that alters its state, rewards are calculated for each actuator device using the most recent recommended state (A_i) and the current user-selected state (U_i) for that device in this event. If the device state remains as recommended by its corresponding controller agent, we can say that the implicit feedback is positive; otherwise, it is negative. For example, if a user turns off the bedroom light, this indicates disagreement with the agent's earlier recommendation to keep it on, resulting in negative implicit feedback.

4.2. Defining the Markov decision process (MDP)

To formally model our DRL-based RS problem, we use a MDP. This section details the key components of our MDP model, including the definition of states, the set of possible actions, and the reward function used to define the relationship between the agents in our RS and the *Environment*.

4.2.1. States

To ensure a robust state representation and handle the complexity of heterogeneous smart devices, we explicitly define a hierarchical structure for device attributes. Before detailing the specific state subsets, we categorize these attributes into two fundamental types:

- **Primary State ($S_{i,p}$):** Represents the device's fundamental operational status, typically a binary value (e.g., 'ON'/'OFF') or a discrete

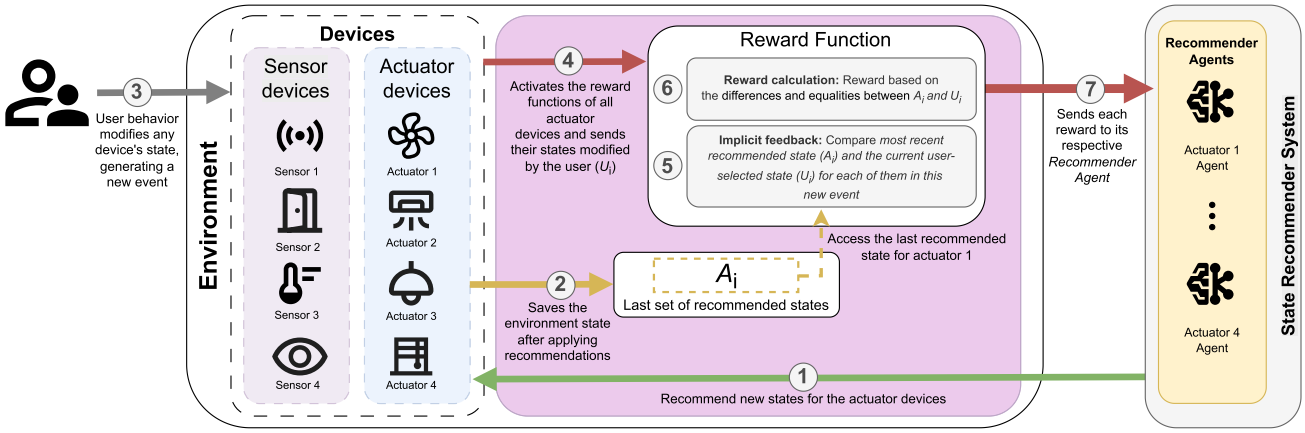

Fig. 3. Modeling implicit feedback by converting user interactions into rewards for recommender agents.

activation mode. It acts as a gatekeeper; secondary states are functionally relevant only when the primary state is active.

- **Secondary States ($S_{i,s}$):** Represent the specific configuration parameters, such as temperature setpoints, light intensity, or color values, which depend on the device being in an active primary state.

Based on this hierarchical definition, the system constructs the complete state vector. Each record processed by the agent is represented by the union of three state subsets $S = S^T \cup S^S \cup S^A$, where:

- S^T represents the temporal states;
- S^S represents the sensor device states;
- S^A represents the actuator device states.

The specific components of these subsets are detailed below:

- **Temporal states**: a subset of the *Environment* state referring to the states extracted from the timestamp, representing the temporal context for learning. Our proposal includes three temporal variables:
  - **Day of the week**: Numbers from 0 to 6 referring to weekdays, where 0 represents Monday, and 6 represents Sunday.
  - **Hour-level interval**: Values from 0 to 3, as the day is divided into four 6-hour intervals, where 0 represents the interval between 00:00 and 06:00.
  - **Minute-level interval**: Values from 0 to 143, since the day is divided into 144 intervals of 10 min each, where 0 represents the interval between 00:00 and 00:10.

  The set of temporal states is represented by:

$$\begin{aligned} WD &= \{sin(\text{weekday}), cos(\text{weekday})\}, \\ HI &= \{sin(\text{hour-interval}), cos(\text{hour-interval})\}, \\ MI &= \{sin(\text{minute-interval}), cos(\text{minute-interval})\}, \\ S^T &= WD \cup HI \cup MI. \end{aligned} \tag{4}$$

  The use of sine and cosine provides a representation that reflects the cyclical nature of temporal variables. It allows models to capture sequential and temporal patterns (Kazemi et al., 2019; Vaswani et al., 2017). For example, the distance between 11 PM and midnight is only one hour, but if represented in their original form, this distance would be 23 (23 − 0).
- **Sensor device states**: It is also a subset of the *Environment* state, represented by:

$$S^S = \{S_1^S, S_2^S, \dots, S_p^S\} \subseteq S. \tag{5}$$

  The state of each sensor device S_i^S is also a subset, containing the primary state and, if present, secondary states:

$$S_i^S = \{S_{i,p}^S\} \cup \{S_{i,s_1}^S, S_{i,s_2}^S, \dots, S_{i,s_k}^S\}, \tag{6}$$

  where $S_{i,p}^S$ is the primary state of sensor i, and S_{i,s_j}^S are its secondary states, with $j = 1, 2, \dots, k$ and $k \geq 0$.
- **Actuator device states**: It is another subset of the *Environment* state, represented by:

$$S^A = \{S_1^A, S_2^A, \dots, S_m^A\} \subseteq S. \tag{7}$$

  As with the sensor devices, each actuator state S_i^A is a subset composed of a primary state and, optionally, secondary states:

$$S_i^A = \{S_{i,p}^A\} \cup \{S_{i,s_1}^A, S_{i,s_2}^A, \dots, S_{i,s_k}^A\} \tag{8}$$

  where $S_{i,p}^A$ is the primary state of actuator i, and S_{i,s_j}^A are the secondary states of actuator i, with $j = 1, 2, \dots, k$ and $k \geq 0$.
- **Environment state**: The complete state, which includes temporal states (S^T), sensor device states (S^S), and actuator device states (S^A). It is the result of the processing of the house model. This processing results in an *Environment* state represented by:

$$S = S^T \cup S^S \cup S^A. \tag{9}$$

Finally, regarding the dimensionality of the resulting state space, we rely on the Dynamic Integer Encoding performed by the *Data Processing Agent* (Fig. 2). By mapping each unique tuple of primary and secondary attributes to a single scalar identifier, the dimensionality of the state representation for any device i in the neural network input is always 1. We use this approach to mitigate the high dimensionality inherent to complex IoT devices.

4.2.2. Actions

The action space of each controller agent is dynamic and increases with each novel state observed for the corresponding device. For example, turning on a lamp can be recommended only if there is prior evidence that it was on. Thus, agents need not consider all possible device actions, but only those previously selected by residents.

Based on state encoding, where a single numeric code represents a set of device states. The actions taken by agents are represented using the same numeric codes, which are later decoded to enable state transitions and reward computation in the subsequent event. Fig. 4 shows that the action space grows as the device transitions to a previously unseen state. Smaller action spaces tend to help agents learn faster and make more accurate decisions by reducing the number of options they need to explore and evaluate.

4.2.3. Reward function

With our implicit feedback already modeled, the reward function is computed as the difference between the recommended states and the user's modifications. Our reward function was designed to prioritize the actuator's primary states, incorporate the secondary-state difference, and normalize the reward. The recommendations generated by the *Recommender Agents* will be evaluated through the value returned by the reward function,

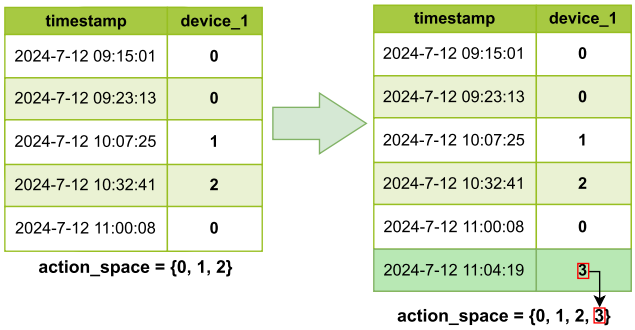

Fig. 4. Dynamic action space.

$$
\begin{aligned}
&R_{i,p} = 1(A_{i,p} = U_{i,p}),\\
&R_{i,s_j}(A_{i,s_j}, U_{i,s_j}) = \begin{cases} 1(A_{i,s_j} = U_{i,s_j}), & \text{if } U_{i,s_j} \text{ is categorical,} \\ 1 - \sqrt{\left|\frac{A_{i,s_j} - U_{i,s_j}}{\max(A_{i,s_j}, U_{i,s_j})}\right|}, & \text{if } U_{i,s_j} \text{ is numerical} \end{cases},\\
&R_{i,s} = \sum_{j=1}^{p} R_{i,s_j},\\
&R_i = R_{i,p} \cdot \left(\frac{R_{i,p} + R_{i,s}}{n_i} + 1(A_i = U_i) \right),
\end{aligned} \tag{10}
$$

where the reward for each new event is determined by the action executed in the previous event. This reward function consists of four steps:

1. **Primary State Reward ($R_{i,p}$)** uses an indicator function $1(\cdot)$ to evaluate whether the recommended primary state $A_{i,p}$ by the actuator agent i matches the state chosen by the user $U_{i,p}$ in the next event, returning one if they match (i.e., the agent is correct) and zero otherwise.
2. **Secondary State Rewards (R_{i,s_j})** evaluates the accuracy of the secondary state recommendation A_{i,s_j} against the user-selected state U_{i,s_j}. If U_{i,s_j} is categorical, the reward is one if the values match and zero otherwise. If U_{i,s_j} is numeric, the reward is calculated as $1 - \sqrt{\left|\frac{A_{i,s_j} - U_{i,s_j}}{\max(A_{i,s_j}, U_{i,s_j})}\right|}$.
3. **Total Secondary State Reward ($R_{i,s}$)** the rewards from all secondary states are summed to obtain $R_{i,s}$, reflecting the agent's overall performance regarding secondary states.
4. **Full Reward (R_i)** is the product of the primary reward $R_{i,p}$ and the average of $R_{i,p} + R_{i,s}$ divided by the total number of states n_i of actuator i. If the agent fails the primary state ($R_{i,p} = 0$), the total reward is 0. If the agent correctly identifies the primary state, the total reward is computed over both the primary and secondary states. Additionally, a bonus value $1(A_i = U_i)$ is added when the agent correctly recommends all states (primary and secondary), effectively doubling the reward.

The design of our reward function is strictly derived from the hierarchical nature of smart devices, rather than arbitrary heuristic tuning. The multiplicative structure involving $R_{i,p}$ acts as a logic gate: physically, secondary attributes (e.g., color, intensity) are only realized when the device is active (in the primary state). Therefore, if the agent fails to predict the primary state ($R_{i,p} = 0$), the entire reward is zeroed to prevent the accumulation of 'false positive' rewards. Furthermore, the function avoids tunable hyperparameters (such as weighted sums) by utilizing averages normalized by the number of states (n_i). Finally, the explicit addition of the term $1(A_i = U_i)$ introduces a sparse reward component solely for perfect matches, encouraging the agent to converge towards the exact ground truth rather than settling for suboptimal partial matches.

4.3. Adaptive action selection strategies

The non-stationarity of environments, particularly in smart homes where user routines change significantly over time, poses a substantial challenge for autonomous agents. Following such shifts, an agent utilizing a pre-established action policy often experiences performance degradation and an increased error rate (Ditzler et al., 2015; Padakandla et al., 2020). This issue is compounded by the fact that these settings constitute continuous-horizon environments; consequently, the action-selection function must dynamically adjust its exploration rate. This adjustment, based on the agents' current performance, mitigates the system's tendency toward a purely greedy strategy and ensures the long-term effectiveness of the adopted policy (Carvalho et al., 2025). In response to this problem, we propose two domain-specific action-selection heuristics that adapt to these dynamics, guided by the average daily reward.

In the proposed RS, the average reward $\bar{r}_d$ is computed and stored by each agent at the end of each day. Based on this metric, we adapted both action selection algorithms so that the ϵ value in the *decaying ϵ-greedy* and the τ value in the *softmax* are adjusted according to the average reward received throughout the experiment. These adaptations are designed to increase exploration during low-reward periods and prioritize exploitation during high-reward phases. Essentially, this forces the agent to re-evaluate its choices immediately after a routine change, as the reward tends to drop abruptly in such situations. Based on this approach, we introduced two tailored adaptations of standard action selection algorithms:

- ***Reward-based ϵ-greedy***: In this method, we define maximum and minimum values for ϵ (ϵ_{max}, ϵ_{min}), representing the range in which ϵ can fluctuate. At the end of each experiment day, the average daily reward ($\bar{r}_d$) is calculated. Then, its value is compared to 0.5, and a new ϵ value is set. The higher the $\bar{r}_d$, the lower the new ϵ. In our setup, ϵ_{min} and ϵ_{max} are 0 and 1, respectively. The value of $\bar{r}_d$ always lies between 0 and 2 in both the simple-state and the composite-state approach. Therefore, we limit $\bar{r}_d$ to 1 and compare it to 0.5, which is the average possible reward in both state approachs, when we do not consider the total hit bonus $1(A_i = U_i)$ responsible for making the rewards jump from 1 to 2, as can be seen in Eq. (10). This ensures that the better the agent performs, the more it tends to choose the action it considers best. The next ϵ value is then computed as follows:

$$
\begin{aligned}
&weight = \begin{cases} \epsilon_{max}, & \text{if } \bar{r}_d \le 0.5 \\ \frac{\epsilon_{max}}{2}, & \text{if } \bar{r}_d > 0.5 \end{cases},\\
&new_\epsilon = \max(\epsilon_{min}, weight \cdot (1 - \min(\bar{r}_d, 1))).
\end{aligned} \tag{11}
$$

- ***Reward-based softmax***: In this method, we also define maximum and minimum values, but this time for τ (τ_{max}, τ_{min}), which represent the range in which τ can fluctuate. Since the maximum possible value for $\bar{r}_d$ is 2, we limit it to 1 so that the average daily reward influences τ only between τ_{max} and τ_{min}. This value directly impacts the new τ in a proportional manner: the greater the $\bar{r}_d$, the greater the new τ, which is calculated as:

$$
\tau = \max(\tau_{min}, \tau_{max} \cdot \min(\bar{r}_d, 1)). \tag{12}
$$

5. Experiment setup

5.1. Real-world dataset

During the experiments, we used a dataset of real records captured in a residential IoT environment with single-resident occupancy. The experimental scenario was confined to a bedroom, designated as the primary location of occupancy due to its dual use for rest, work, and leisure. Data collection spanned 17 days, from August 3, 2024, at 23:40:37 to August 19, 2024, at 08:44:46, resulting in a total of 6,560 recorded events. To capture the environment's dynamics, the space was instrumented with three actuators (a smart bulb, a smart plug connected to a

Table 2
Different types of messages that can be generated by HESTIA.

State approach	Description	Example
Simple	It considers only a single state for each device, focusing on their primary states. For example, a lamp can only be turned on or off.	`{'status': [ {code: switch, value: ON} ]}`
Composite	It considers more than one state for each device. For example, in addition to the lamp being able to be turned on and off, its light intensity, lamp color, and other secondary states can also be modified.	`{'status': [ {code: switch, value: ON}, {code: temp_set, value: 24}, {code: mode, value: WIND} ]}`

computer, and a fan) and sensors to monitor presence, window opening, temperature, and humidity.

The monitored routine, which comprises the behavioral context, presents distinct temporal patterns. On weekdays, the resident is typically absent between 09:00 and 19:00, maintaining a nightly rest between 23:00 and 00:00. Conversely, on weekends, room occupancy intensifies with prolonged computer usage, interspersed with brief absences for household activities in other rooms. This variability provides a realistic scenario for validating the proposed algorithms.

Prior to use in the experiments, the raw data were processed and normalized. This stage included excluding irrelevant technical reports, such as battery levels, and reconstructing the device state. Reconstruction was necessary to convert the original protocol, which recorded only partial changes (deltas), into complete state vectors, ensuring the contextual integrity of each processed event.

5.2. Synthetic datasets

We employed Home Environment Simulator Targeting Inhabitant Activities (HESTIA) (dos Santos et al., 2025), a simulator for resident-oriented smart environments, to generate two synthetic datasets. This enabled us to extend the evaluation period and simulate controlled routine shifts, assessing the approach's adaptability beyond the temporal constraints of real-world capture. This simulator can generate data for smart environments focused on user interaction. Data generation is based on a graph that specifies the distribution of rooms and indicates which are active based on user interaction. For each user, it is possible to define an activity routine for each day of the week and their preferred device state for each activity.

The simulator data provides device interaction data using two approaches: simple-state, which includes only the device's primary state (e.g., switch), and composite-state, which includes the primary state plus additional secondary states (e.g., temp_set and mode). Examples of both message formats are detailed in Table 2.

Fig. 5 shows the simulated environment used for data generation, consisting of a house with nine rooms: a living room, a hallway, a kitchen, an office, two bedrooms, two bathrooms, and a balcony. Across this environment, 26 smart devices, including nine sensors and 17 actuators, are distributed throughout the rooms. Based on this setup, we created three routines in Hestia to generate data under different occupancy configurations and resident activities.

- **Routine A**: Coexistence routine of two people with different work and study routines. Both share the same space, but their daily activities occur at specific times and places. One resident works full-time in person, while the other works remotely and studies at night, creating a complementary flow of activities throughout the day. User 1 works in person, and his routine is dictated by work. From Monday to Friday, he wakes up early, leaving for work at 7:30 AM and returning late in the afternoon, around 5:00 PM, with the rest of the evening dedicated to rest. His sleep period begins earlier, approximately 9:15 PM. User 2, due to remote work and nighttime study, has a more flexible routine. From Monday to Friday, he starts work in his home office at 8:15 AM, with a lunch break. The workday ends at 5:10 PM. In the evening, the study routine begins with leaving for college at 6:30 PM, returning around 11:00 PM, and then going to bed at 11:20 PM.
- **Routine B**: This routine describes a living environment for three people with distinct work and study routines. While two residents work full-time in person, the third divides their day between an internship and evening studies, generating a flow of complementary activities throughout the day. User 1 and User 2 both work in person but have slightly different morning routines. User 1 leaves for work early, at 7:30 AM, and returns at 5:00 PM. User 2 includes a trip to the gym before leaving for work at 8:20 AM and returning at 5:50 PM. Both end their workdays early, going to bed around 9:15 PM and 9:10 PM, respectively. User 3's routine is split between his morning internship and evening college. He leaves for his internship at 7:20 AM and returns at noon. The afternoon is dedicated to rest and leisure activities. He leaves for college again at 6:20 PM and only returns home at 10:50 PM, with his sleep period beginning later, around 11:10 PM. His routine contrasts with that of the other residents, who end their workdays earlier.
- **Routine C**: This routine describes the environment shared by two residents who work in person during the day, with well-defined schedules and activities. Their routines are similar in structure, with slight variations in departure and return times, demonstrating a distinct flow of activities throughout the day. User 1 follows a more regular pattern: he wakes up early, leaves for work at 7:30 AM, returns at 5:00 PM, and goes to bed around 9:15 PM. User 2 incorporates an extra activity into his morning routine. He wakes up at the same time but goes to the gym before leaving for work at 8:20 AM. Consequently, he returns home at 5:50 PM and goes to bed at 9:10 PM, slightly earlier than User 1.

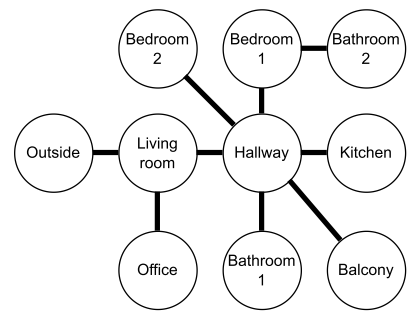

Fig. 5. Graph of the house model used for routine generation. Vertices represent the rooms, and edges represent the connections between them.

It is essential to note that these routines apply to weekdays (Monday through Friday). The specified time ranges are approximate and may vary slightly due to changes in the start and end times of activities. Furthermore, weekend activities vary widely across users, typically including leisure pursuits such as walking, exercising, playing video games,

watching TV, and playing soccer. By simulating these routines, HESTIA generated two distinct datasets, each spanning 90 days and documenting the transition between three different routines. Both datasets share the common characteristic of simulating usage data in a smart environment with routine changes over time.

- **Dataset AC**: Records 20,847 events with 8 sensors and 13 actuators, spanning 90 days, divided into two routines: A (days 1 to 60) and C (days 61 to 90). Initially, one of the two residents worked remotely but began working in person after 60 days, which disrupted the routine and enabled analysis of the impact on the recommendation system's performance.
- **Dataset BC**: With 26,238 events, 9 sensors, and 17 actuators, it reflects the occupancy of three residents during the first 60 days of Routine B. After this period, the departure of one resident reduced the number of residents to two, leading to the transition to Routine C and allowing the assessment of the impact of the change in the number of users.

With these two datasets generated from the simulation, it is possible to analyze changes in speakers' routines within the same environment, thereby enabling evaluation of the proposed recommendation system's performance.

5.3. Validation metrics

To validate our proposal, we must demonstrate both the average of the presented validation metric and its daily values, thereby showing the evolution of learning over time.

5.3.1. Average daily reward

In RL research, the average reward is used as the primary validation metric most of the time. As rewards are fundamental to both agents' learning and their operation, analyzing them is essential to understanding the outcomes.

The calculation of the average reward is performed daily,

$$\bar{r}_d = \frac{1}{T}\sum_{t=1}^{T} r_t, \tag{13}$$

where the daily average for the experiment day d is equal to the sum of the rewards from all time steps t that occurred on day d divided by the total number of time steps T during that day. This analysis enables the assessment of agent performance across experimental days and the comparison of performance variations across different days of the week. It also characterizes the agents' behavior in response to shifts in residents' routines, which induce corresponding changes in device-state patterns.

5.3.2. Daily hierarchical hamming score

In this study, we introduce Hierarchical Hamming Score (HHS) to evaluate performance in tasks with hierarchically dependent classes, a characteristic often ignored by standard multi-label metrics. Unlike the traditional Hamming Score, which treats all labels independently, our context involves physical constraints in which secondary states (e.g., color, intensity) are relevant only when the primary state (e.g., power switch) is correctly engaged. A standard metric might award partial credit for correct secondary parameters even if the device is wrongly turned off, overestimating the agent's utility. To address this, HHS imposes a strict penalty: if the primary state prediction is incorrect, the score for that instance is set to zero, regardless of secondary matches, ensuring the metric aligns with the user's operational intent.

$$HHS_d = \frac{1}{T}\sum_{t=1}^{T}\begin{cases} 0, & \text{if } Y_{t,0} \neq \hat{Y_{t,0}} \\ \frac{1}{n}\sum_{i=0}^{n}\delta(Y_{t,i}, \hat{Y_{t,i}}), & \text{otherwise} \end{cases}, \tag{14}$$

where d is the experiment day, n is the total number of states (including primary), T is the total time steps, $Y_{t,0}$ and $\hat{Y_{t,0}}$ are the expected and recommended primary states, and $Y_{t,i}$ and $\hat{Y_{t,i}}$ correspond to the secondary states. This calculation ensures that a "total success" requires the correct prediction of the entire hierarchy, while a 'partial error' is possible only if the fundamental action (primary state) is incorrect.

5.4. Supervised learning baseline: Online behavior cloning

To rigorously assess the specific contribution of the sequential decision-making formulation provided by the DRL agents, we implemented an online Supervised Learning (SL) baseline, specifically a Behavior Cloning agent. Unlike RL, which aims to maximize cumulative future rewards through trial-and-error, this baseline treats the smart home automation task as a multi-class classification problem. Its objective is to minimize the immediate prediction error between the recommended action and the user's actual action using Cross-Entropy Loss, effectively cloning the user's behavior pattern without forward-looking planning.

The selection of this specific baseline is grounded in the need to isolate the algorithmic performance from the neural network's capacity. By comparing DRL agents with a Supervised agent that shares the same feature extraction capabilities, any performance divergence can be attributed to the learning paradigm (RL vs. Supervised) rather than the model architecture.

To ensure a fair comparison, the SL agent utilizes the exact same Deep Neural Network (DNN) architecture as the DQL and DSGnSARSA agents. The network consists of three fully connected hidden layers with 64, 128, and 64 units, respectively, using Rectified Linear Unit (ReLU) activation functions. The output layer size is dynamically updated to match the number of available actions in the smart home environment.

The training is performed incrementally online. Similar to the RL agents, the SL agent stores experiences in a replay buffer and samples a batch of size 100 uniformly to mitigate catastrophic forgetting and temporal correlations. The optimization is performed using the *Adam* optimizer.

Regarding the hyperparameter configuration, we conducted a grid search to identify the optimal settings for the baseline, mirroring the rigorous tuning process applied to the DRL agents. We evaluated the Learning Rate over the set $\{10^{-3}, 10^{-4}\}$. The configuration with learning rate equals 10^{-3} consistently yielded the highest HHS across all experimental scenarios and was therefore selected for the final comparison. All other structural parameters, such as the batch size of 100 and the use of the Adam optimizer, remained fixed to match the DRL agents' configuration.

The fundamental distinction between the proposed DRL agents and this baseline lies in their respective optimization objectives. While the DRL agents aim to maximize cumulative future rewards using the Mean Squared Error (MSE) loss on the Temporal Difference error, the Behavior Cloning agent approaches the task as a multi-class classification problem. It optimizes the network using the *Cross-Entropy Loss* function to minimize the immediate divergence between the predicted action probabilities and the ground-truth user action. This formulation forces the agent to strictly imitate the user's immediate behavior without the exploratory or forward-looking planning capabilities inherent to the RL paradigm.

5.5. Hyperparameter tuning

During the experiment, the two algorithms presented in Section 3.2 are used: DQL and DSGnSARSA. Each of them employed two state approaches and utilized the three datasets listed in Sections 5.1 and 5.2. Employing a grid search strategy to find the best hyperparameters, we evaluated 48 combinations for DQL and 32 combinations for DSGnSARSA. Each combination was executed 3 times, using a different random seed to ensure robust results, for a total of 1440 experimental trials.

The overall metrics are computed by averaging the validation metrics across runs with three random seeds for each combination of algorithm, approach, and hyperparameter set. Running the algorithm once

Table 3
Optimal hyperparameters for the DQL algorithm on each dataset and state approach.

State approach	Dataset	Discount	Experience type	Action selector	Learning rate
simple	AC	0.90	random	reward-based softmax	0.001
	BC	0.90	random	reward-based ϵ-greedy	0.001
	real-world	0.95	random	softmax	0.001
composite	AC	0.90	random	reward-based ϵ-greedy	0.001
	BC	0.95	random	reward-based ϵ-greedy	0.001
	real-world	0.9	random	decaying ϵ-greedy	0.001

Table 4
Optimal hyperparameters for the differential semi-gradient n-step SARSA algorithm on each dataset and state approach.

State approach	Dataset	Action selector	Learning rate	n
simple	AC	softmax	0.0001	1
	BC	softmax	0.0001	1
	real-world	reward-based ϵ-greedy	0.0001	1
composite	AC	softmax	0.0001	1
	BC	decaying ϵ-greedy	0.0001	1
	real-world	decaying ϵ-greedy	0.001	1

may yield metrics affected by random variation that do not accurately represent the average performance.

The experiment's hyperparameters determine how agents learn and make decisions. We present the hyperparameters and brief explanations of the values tested during the experiments.

- **For both algorithms**:
  - Action selection algorithm: As discussed in Sections 3.3 and 4.3, we need to specify how the algorithms select actions for the agents. For this experiment, we limited our testing to 4 of these action selection algorithms: *decaying ϵ-greedy*, *softmax*, *reward-based ϵ-greedy*, and *reward-based softmax*.
  - Learning rate: This hyperparameter determines the step size for updating the neural network weights during training. A higher rate yields larger updates, which can lead to oscillation. In contrast, a lower rate yields smaller, more precise updates, thereby improving stability but extending training time. Values like *0.001* and *0.0001* were tested.
- **For DQL only**:
  - Experience type: Can be *random*, always providing a random sample containing n elements from the memory buffer, or *last*, providing the last n elements inserted into the memory buffer;
  - Update frequency: We define the agent update frequency as 10. That is, the DQL agents undergo a new training cycle every 10 timesteps.
  - Discount (γ): This hyperparameter quantifies the importance of future rewards. A value closer to 1 prioritizes long-term gains, weighing future rewards almost as much as present rewards. Conversely, a value closer to 0 emphasizes immediate rewards. Tested values included *0.85*, *0.9*, and *0.95*.
- **For DSGnSARSA only**:
  - n: In n-step SARSA, this parameter defines the number of future steps used for calculating returns. A smaller n (e.g., 1) favors immediate rewards, while a larger n (e.g., 5) considers longer future sequences, potentially speeding learning but increasing variance. Values *1*, *2*, *3*, and *5* were tested.

Additionally, we specified hyperparameters that modify the algorithms executed within the Agents, along with the values to experiment with them. The learning rate, discount, and n value hyperparameters are used in one or both of the tested algorithms and modify how these algorithms perform Agent learning.

Tables 3 and 4 show the hyperparameters tested for each of the two experimented algorithms. Additionally, the best hyperparameter combination for each algorithm on each dataset is marked in bold. To define the optimal hyperparameter combinations for each approach, algorithm, and dataset, we used the average daily reward across all actuator devices in the house over all experimental days. This is because, unlike how we calculate the HHS, which only considers agents' hits and errors for each state, the reward also considers the distance between numerical states in cases where agents recommend a state with partial error.

6. Results

The discussion is organized as follows: first, we report the outcomes of both algorithms under the simple-state approach across the three datasets, focusing on the HHS metric. Next, we extend the analysis to the composite-state approach, which also includes the average reward metric. All reported results correspond to the hyperparameter configurations that achieved the highest average rewards, as summarized in Tables 3, 4, and in the Section 5.4.

6.1. Simple-state approach

As discussed earlier, since this approach considers only primary device states, the action space is smaller and less complex than in the alternative setting; consequently, the reward and HHS reduce to scalar quantities, making them directly comparable. The reward value will always be double the HHS (based on Eqs. (14) and (10)). Therefore, for the simple-state approach, it is sufficient to present a single metric, and we opt for the daily HHS.

During the experiments on the AC dataset, we obtained daily HHS values, which are presented in Fig. 6. The DQL algorithm yields an overall average HHS of approximately 96.69%, whereas the DSGnSARSA algorithm achieves a slightly better performance of approximately 97.59%. Meanwhile, the Supervised baseline (Behavior Cloning) achieved a slightly lower HHS of 94.13%. The graphs in Fig. 6 represent both validation metrics presented in this chapter.

In the experiments conducted on the BC dataset, the best hyperparameter combinations achieved average rewards and daily HHS (Fig. 7). It can be observed that when using the same simple-state approach with the BC dataset, the DSGnSARSA algorithm again performs slightly better than DQL, with approximately 97.98% and 96.84% of HHS, respectively. The supervised baseline achieved an HHS of 94.74%.

Figs. 6 and 7 indicate that both algorithms, when tested on synthetic dataset scenarios, showed a performance drop on the day the routine was changed (day 60 of the experiment). However, they eventually converged to their previous performance levels. Notably, DQL demonstrated greater difficulty in this recovery process compared to DSGnSARSA. Furthermore, as shown in Fig. 6, during experiments on the AC dataset, the DSGnSARSA algorithm achieved a higher HHS than DQL on 84 of 90 days. In the remaining days, DQL showed superior performance on 4 occasions, while the Supervised baseline did so on 2 days. The baseline's distinct advantage in the initial phase is attributed to the absence of the exploration mechanisms inherent to Reinforcement Learning, allowing it to immediately exploit observed patterns without the initial performance cost of trial-and-error experimentation. In Fig. 7, a similar trend is observed: the Supervised baseline achieved the highest metrics

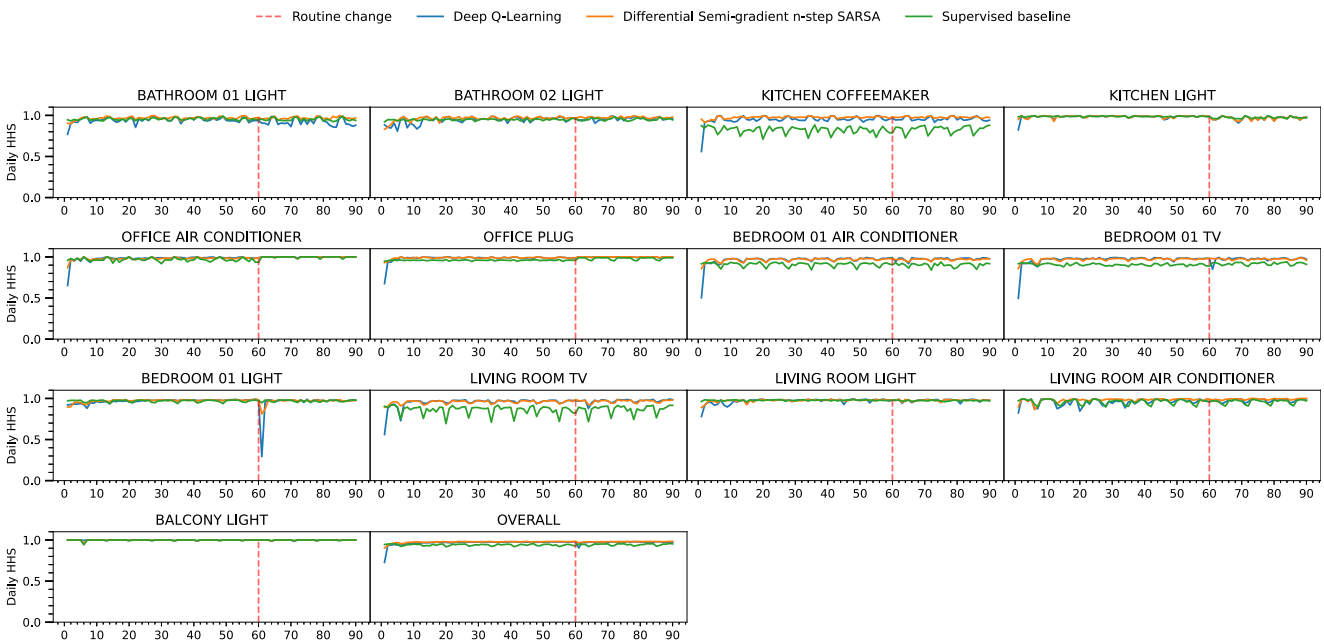

Fig. 6. Daily HHS for each of the actuator devices in the AC dataset experiment using the simple-state approach. The graph titled "OVERALL" shows the average HHS across all devices for each day of the experiment.

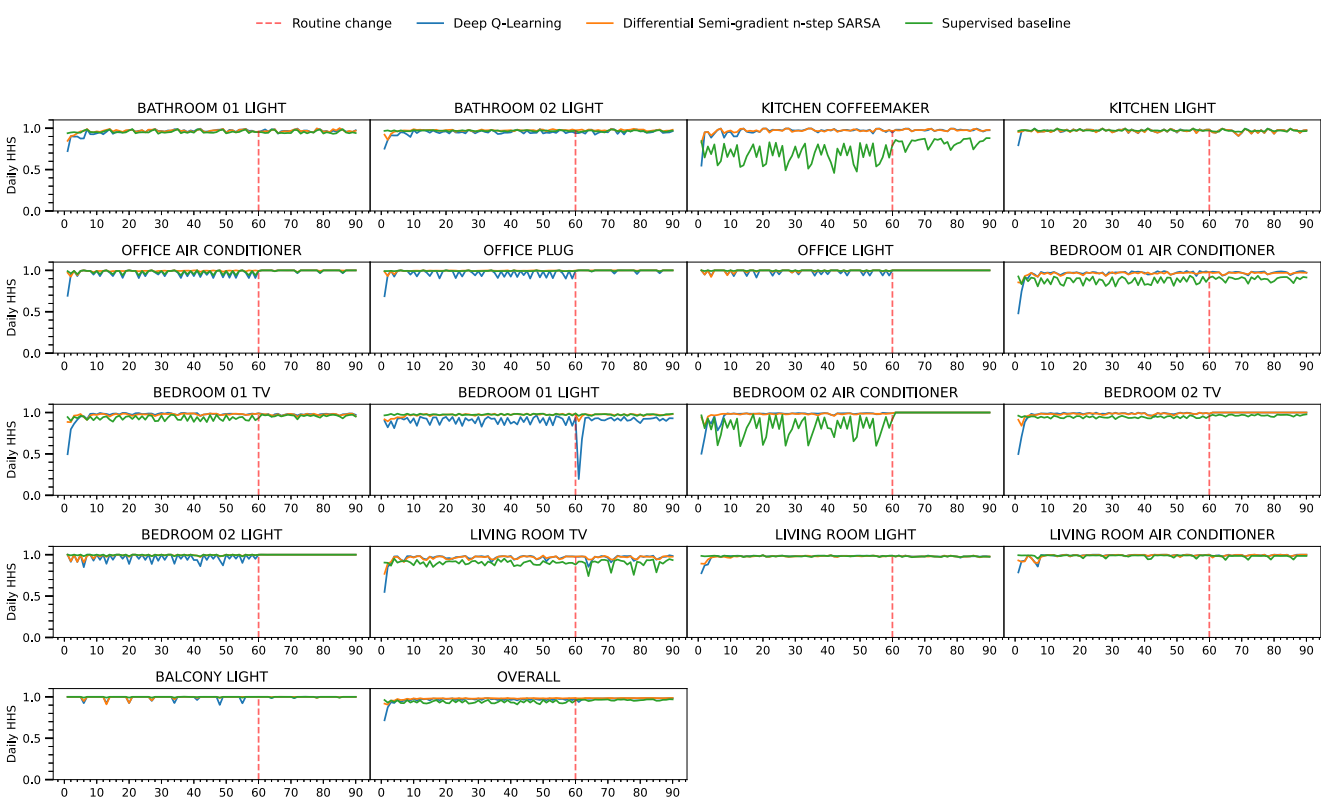

Fig. 7. Daily HHS for each of the actuator devices in the BC dataset experiment using the simple-state approach. The graph titled "OVERALL" shows the average HHS across all devices for each day of the experiment.

during the first 2 days, while DQL surpassed the other models on only a single day, with DSGnSARSA maintaining superior performance for the remainder of the experiment.

On the other hand, the results obtained from the Real-world dataset show a reversal of the performance trend observed in previous experiments, as illustrated in Fig. 8. In this scenario, DQL obtained an overall average HHS of approximately 97.12%, while DSGnSARSA reached approximately 95.93%. Both reinforcement learning approaches achieved higher average HHS values than the Supervised baseline, which recorded 83.67%. Moreover, the DQL algorithm achieved a higher daily average HHS than DSGnSARSA on all 17 days of the experiment. Although the experiment covered a shorter period, these results

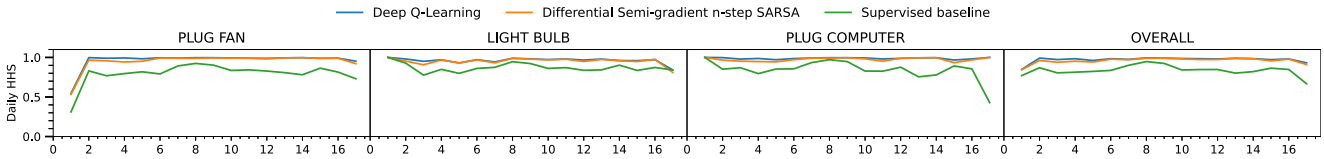

Fig. 8. Daily HHS for each of the actuator devices in the real-world dataset experiment using the simple-state approach. The graph titled "OVERALL" shows the average HHS across all devices for each day of the experiment.

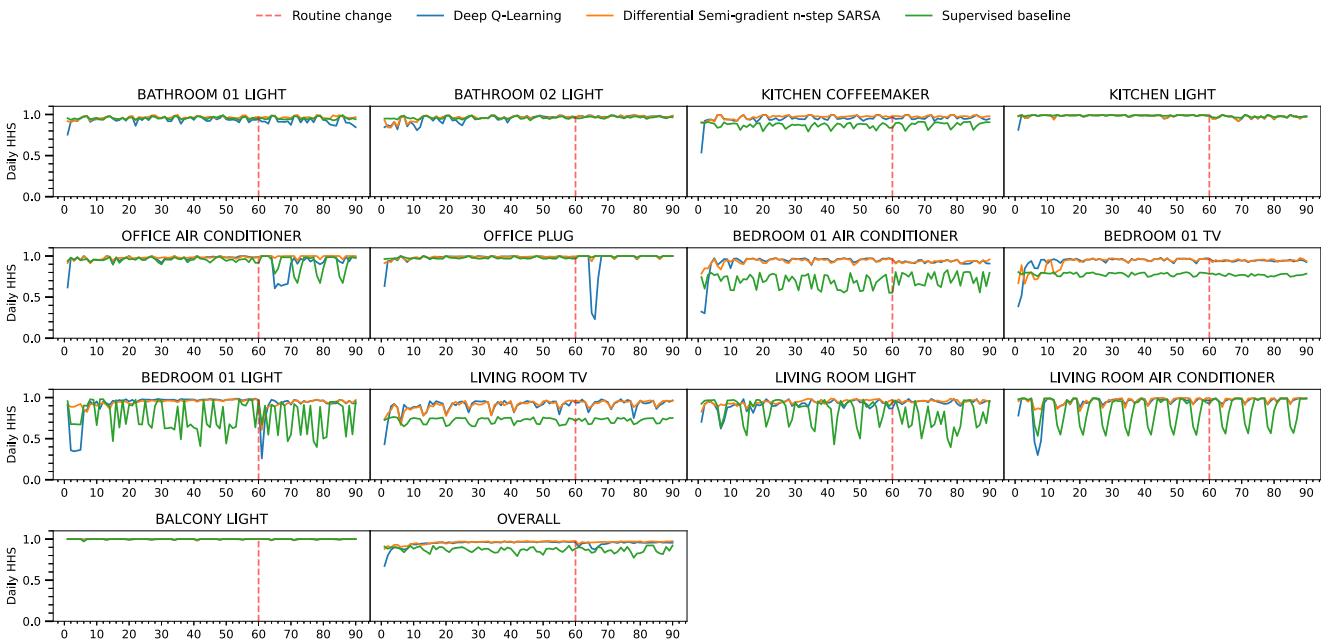

Fig. 9. Daily HHS for each of the actuator devices in the AC dataset experiment using the composite-state approach. The graph titled "OVERALL" shows the average HHS of all devices for each day of the experiment.

indicate that DQL adapted more effectively to real-world usage patterns than DSGnSARSA. This outcome may be partially attributed to the specific characteristics of this dataset: representing a short-term capture of actual usage, it contains no abrupt routine changes. Consequently, DQL was not exposed to the adaptation challenges in which it previously showed a worse recovery than DSGnSARSA.

6.2. Composite-state approach

The composite-state approach, in which most actuators have one or more secondary states, introduces greater complexity. This results in a wider range of state combinations than the simpler approach, leading to expectedly inferior performance. The HHS curves in Fig. 9 indicate that, in the AC Dataset under the composite-state approach, DSGnSARSA obtained higher HHS values than the other methods in most evaluated days. Specifically, it achieved an overall HHS of approximately 95.96%, while the DQL algorithm reached 94.55%, both exhibiting a substantial lead over the Supervised baseline, which achieved an average of 87.17%.

The same behavior is observed in Fig. 10, when applying our system with the composite-state approach to the BC dataset. From the HHS perspective, it is important to note that the performance advantage of the DSGnSARSA algorithm over DQL increases even further. While the overall average HHS for the former is 95.79% and that of the latter is approximately 94.30%, both notably surpassing the Supervised baseline, which attained 90.12%. Additionally, the "OVERALL" graph makes it clearer that, from the HHS perspective, the performance difference between the two algorithms increases during experiments with the BC dataset.

Regarding the results from the Real-world dataset using this approach, Fig. 11 shows that both algorithms performed worse overall than in the synthetic experiments. In this scenario, DQL achieved an average HHS of 92.53%, while DSGnSARSA reached 92.38%. The Supervised baseline obtained a lower average HHS of 74.89%.

In the AC dataset experiments, DSGnSARSA achieved the highest average daily reward on 78 of the 90 evaluated days, while DQL led on 11 days and the Supervised baseline on 1 day.

Meanwhile, during the experiments with the BC dataset, we noticed that the pattern identified by HHS in the previous subsection repeats: DSGnSARSA surpassed the others on 68 out of 90 days, with DQL leading on 16 days and the Supervised baseline on 6 days.

In the real-world dataset, DQL achieved the best HHS value on 12 of the 17 evaluated days, while DSGnSARSA achieved the best HHS value on the remaining 5 days. The Supervised baseline did not obtain the best daily HHS in this dataset. The stable nature of this short-term dataset likely masked DQL's known adaptation weaknesses. Without abrupt routine changes, DQL was not subjected to the recovery scenarios where DSGnSARSA typically proves superior.

Using the overall average rewards, we can better compare the two RL algorithms tested. In Fig. 12, we observe the average daily reward results when applying the proposed recommendation system to the AC dataset with the composite-state approach. The DQL algorithm continues to perform slightly worse than DSGnSARSA. While the former achieved an overall average reward of approximately 1.8866, the latter achieved 1.9177. This seems to have occurred because the agents had more difficulty converging at the beginning of the experiment than after the routine change, suggesting a cold-start problem. Another important observation is that, after the routine change on day 60, performance

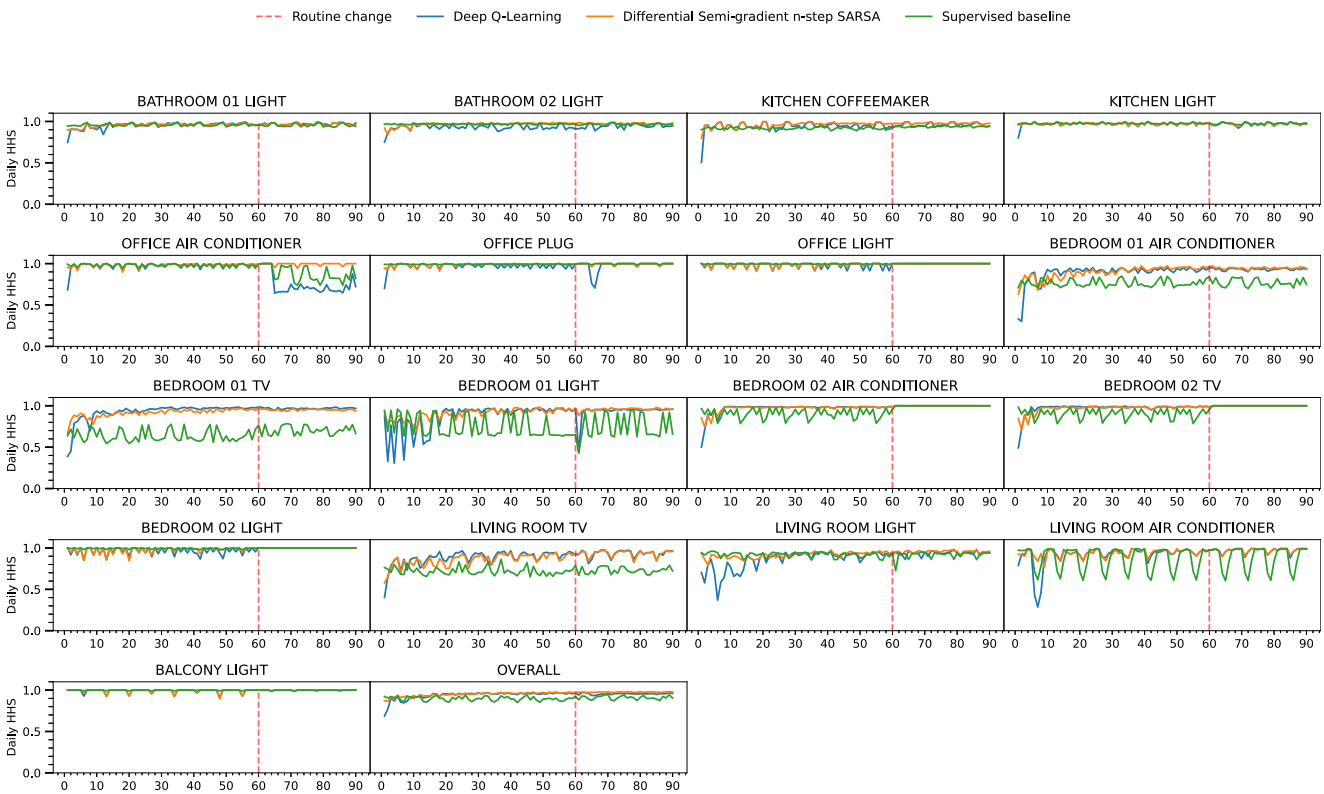

Fig. 10. Daily HHS for each of the actuator devices in the BC dataset experiment using the composite-state approach. The graph titled "OVERALL" shows the average HHS of all devices for each day of the experiment.

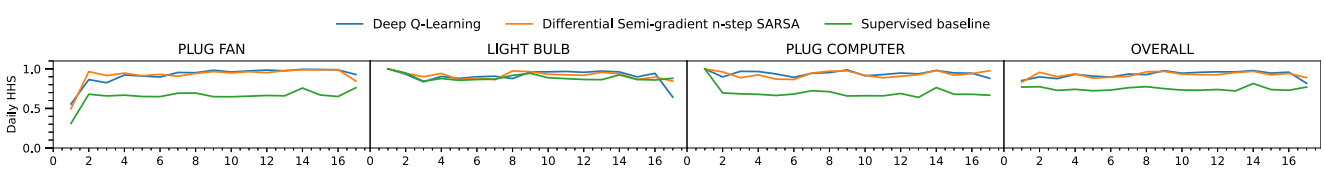

Fig. 11. Daily HHS for each of the actuator devices in the real-world dataset experiment using the composite-state approach. The graph titled "OVERALL" shows the average HHS of all devices for each day of the experiment.

increases slightly for both algorithms, in contrast to the simple-state approach on the same dataset.

In the experiments on the BC dataset shown in Fig. 13, the superiority of DSGnSARSA over DQL becomes even clearer. Comparing the overall average rewards, we have approximately 1.9136 and 1.8776, respectively. A sharp drop in DQL's performance on the "OFFICE AIR CONDITIONER" device after day 60 of the experiment is also observable. However, when we examine the "OVERALL" graph and the overall average across all devices, the phenomenon observed in the AC dataset also appears here, with both algorithms showing slight performance improvements after day 60 of the experiment.

Finally, the results for the average daily reward on the Real-world dataset are presented in Fig. 14. Consistent with the observations for the average daily reward, both algorithms performed worse than in the synthetic experiments, as we also observed with the HHS. In terms of average reward, DQL achieved a value of 1.8427, while DSGnSARSA obtained 1.8397.

In summary, the analysis of average daily rewards reveals a distinct performance dichotomy that depends on the environment's nature. In synthetic scenarios (AC and BC), where routine changes and long-term dependencies are prevalent, DSGnSARSA demonstrates superior stability and higher cumulative returns, effectively mitigating the cold-start problem better than DQL. Conversely, in the Real-world dataset, the data's stability and short duration appear to favor the DQL approach, albeit by a narrow margin. This suggests that while DSGnSARSA is the more robust candidate for long-term, dynamic deployments, DQL remains competitive in shorter, more stable operational windows.

6.3. Statistical analysis

To enable a fair and standardized comparison between the different models evaluated, Reinforcement Learning algorithms (DQL, DSGnSARSA) and the supervised approach (baseline), the HHS metric was adopted as the primary performance indicator. The choice of HHS is justified by its being a cross-sectional metric that quantifies the quality of the decision policy independently of the learning paradigm, providing a common basis for statistical inference. The evaluation was conducted on a total universe of $N = 394$ independent samples. This sample size results from aggregating three distinct datasets (two simulated with 90 days each and one real with 17 days), evaluated under two reward modeling approaches (Simple and Composite), yielding $N = (90 + 90 + 17) \times 2 = 394$.

From a global perspective, the statistical analysis revealed significant differences between the models. The non-parametric *Kruskal-Wallis* test rejected the hypothesis of equivalence between the HHS metric distributions with high confidence ($H = 516.05; p < 10^{-112}$). To establish the performance hierarchy, the Friedman test confirmed the disparity in rankings ($\chi^2 = 586.85; p < 10^{-128}$), indicating the DSGnSARSA algorithm as the best ranked globally, as shown in Table 5, with an aver-

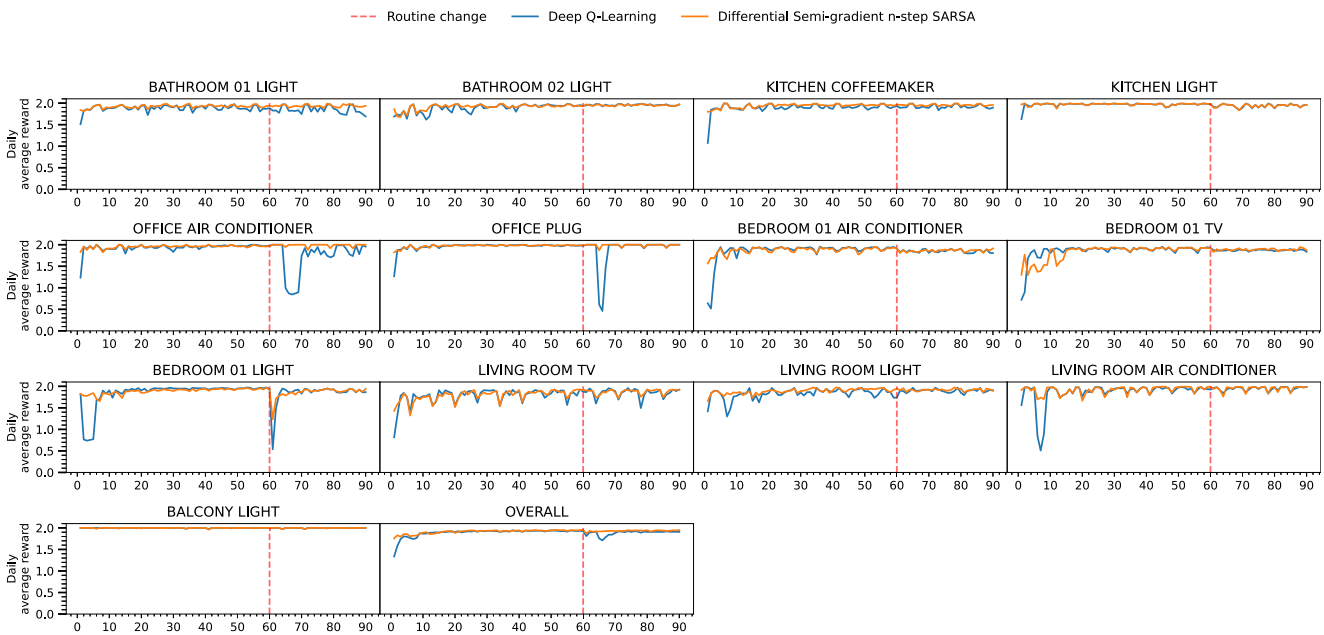

Fig. 12. Average daily reward for each of the actuator devices in the AC dataset experiment using the composite-state approach. The graph titled "OVERALL" shows the average reward of all devices for each day of the experiment.

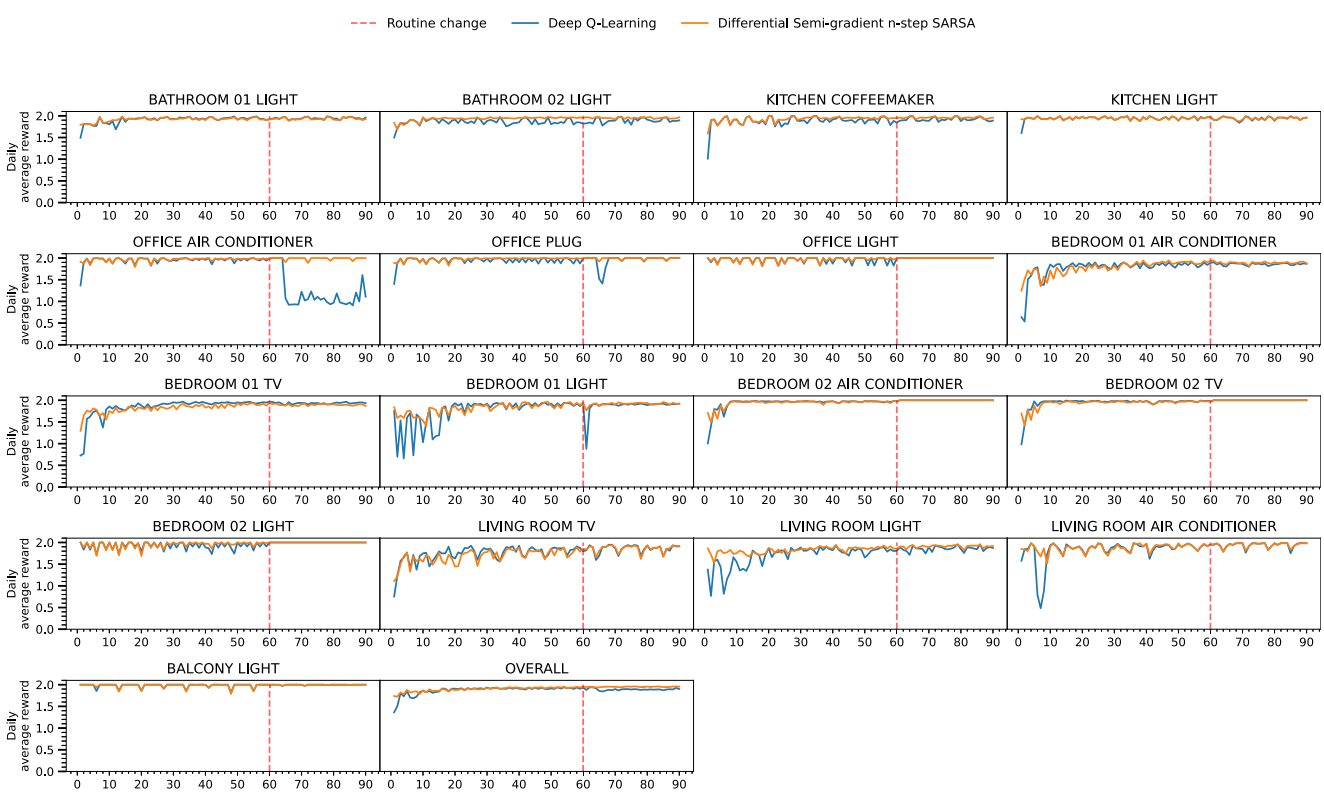

Fig. 13. Average daily reward for each of the actuator devices in the BC dataset experiment using the composite-state approach. The graph titled "OVERALL" shows the average reward across all devices for each day of the experiment.

age rank of 1.19, followed by DQL (1.91) and, lastly, the baseline (2.91). Dunn's post-hoc analysis with Bonferroni correction corroborated that all differences between pairs are statistically significant (Table 6). The direct comparison between the RL agents (DSGnSARSA vs. DQL) showed a p-value of 7.23×10^{-8}, while the superiority of both compared to the SL baseline was confirmed with extreme p-values ($p < 10^{-58}$), validating the effectiveness of Reinforcement Learning for minimizing HHS.

However, a stratified analysis by scenario exposes critical nuances that the overall average, weighted by the larger volume of simulated data ($N = 360$), tends to mask. As detailed in Table 7, in the simulation environments (sets AC and BC datasets), the DSGnSARSA algorithm demonstrated excellent results, achieving average ranks of 1.03 to 1.27 across both simple and composite approaches. These results show that, in controlled environments where the system dynamics conform to

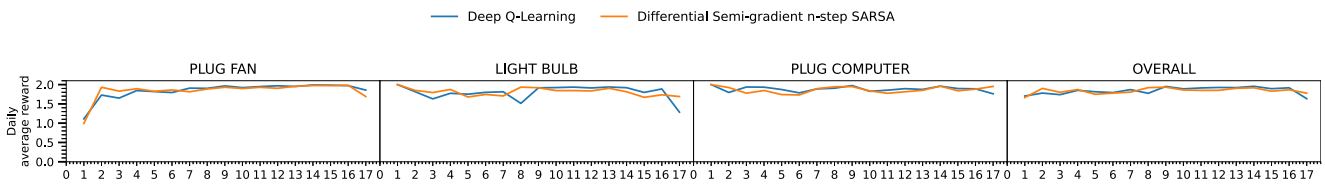

Fig. 14. Average daily reward for each of the actuator devices in the real-world dataset experiment using the composite-state approach. The graph titled "OVERALL" shows the average reward of all devices for each day of the experiment.

Table 5
Global average ranking (Friedman test).

Algorithm	Global Average Ranking	Position
DSGnSARSA	1.19	1st
DQL	1.91	2nd
SL baseline	2.91	3rd

Table 6
P-value matrix of Dunn's Post-hoc test with Bonferroni correction.

	DQL	DSGNSARSA	SL baseline
DQL	–	7.23×10^{-8}	4.05×10^{-59}
DSGnSARSA	7.23×10^{-8}	–	1.86×10^{-105}
SL baseline	4.05×10^{-59}	1.86×10^{-105}	–

Table 7
Detailed average rankings by dataset and approach. The values in **bold** and underlined indicate, respectively, the best and second-best performance (Lowest average ranks) in each scenario.

Dataset	Approach	DQL	DSGNSARSA	SL baseline	Local Winner
AC	Composite	1.93	**1.13**	2.93	DSGnSARSA
	Simple	2.02	**1.07**	2.91	DSGnSARSA
BC	Composite	1.91	**1.27**	2.82	DSGnSARSA
	Simple	2.04	**1.03**	2.92	DSGnSARSA
Real-world	Composite	**1.29**	1.71	3.00	DQL
	Simple	**1.00**	2.00	3.00	DQL

the training assumptions, DSGnSARSA achieved the best performance across the simulated scenarios, as measured by the HHS metric.

Conversely, a significant performance reversal is observed in the real-world environment (Real Data), highlighting the Sim-to-Real gap phenomenon. In this scenario, DQL took the lead, achieving a rank of 1.00 in the simple variant and 1.29 in the composite variant, while the performance of DSGnSARSA degraded, with values between 1.71 and 2.00. This reversal suggests that DQL is more robust to stochastic noise and unmodeled dynamics inherent to the physical world, generalizing better under conditions of practical uncertainty, whereas DSGnSARSA is more sensitive to discrepancies between the simulated model and reality. In all cases, the baseline approach consistently ranked last, reaffirming the superiority of the proposed RL strategies.

6.4. Computational efficiency analysis

To evaluate the computational efficiency of the proposed algorithms, we monitored their execution times in a controlled environment. All experiments were conducted on a workstation equipped with an AMD Ryzen 7 5700X3D processor, 32 GB of RAM, and an NVIDIA GeForce RTX 4060 (8 GB) GPU. The algorithms were implemented using Python and the PyTorch framework. It is important to note that all computations were explicitly executed on the CPU to assess performance baselines without hardware acceleration.

The results in Table 8 indicate distinct performance profiles per agent, driven by dataset volume and algorithmic mechanics. First, the significant disparity in total experimental time between the synthetic datasets (AC and BC) and the Real-world dataset is primarily due to the temporal scope of the data. The synthetic environments simulate 90 days, whereas the Real-world dataset comprises only 16 days.

Regarding the algorithms, the SL baseline consistently demonstrated the highest computational efficiency, achieving the lowest total execution times and step latencies across all datasets. This is attributed to the absence of complex reinforcement learning mechanisms, such as Bellman equation updates, target network synchronization, and future reward estimation, which allows the supervised model to focus solely on the forward and backward passes of the neural network.

Among the RL agents, although DQL often incurs a slightly higher per-step training latency due to processing batches of 100 samples, its total execution time is consistently lower than that of DSGnSARSA. This occurs because DQL performs optimization updates only once every 10 timesteps. In contrast, DSGnSARSA updates its policy at every timestep using the last n records, resulting in a much higher cumulative computational cost despite its lower per-step latency. Finally, the increase in runtime observed in the "composite" approach compared to the "simple" one reflects the complexity of the state space.

In terms of memory usage, the average footprint for all agent models remained extremely low. The Supervised baseline served as the lower bound, requiring approximately 20 KB to store the neural network parameters. Conversely, the DQL agent required approximately 34 KB. Despite sharing the same network architecture, the DQL framework incurs additional overhead due to maintaining the Target Network and the Experience Replay buffer. The DSGnSARSA agents presented an intermediate footprint of around 22 KB. These compact memory requirements, paired with CPU-based execution, highlight the feasibility of deploying the proposed solution on resource-constrained edge devices.

Crucially, the "Train step latency" presented in Table 8 demonstrates the system's capability for real-time, online learning. Since the proposed algorithms perform incremental updates continuously as new interactions occur, this latency represents the time required to incorporate new user feedback into the model immediately. Given that the computational costs for the RL approaches fall within the 1.5–2.3 ms range on CPU-based execution, the overhead of online updates renders itself negligible. Consequently, in a real-world deployment, perceptible latency would primarily result from network infrastructure and hardware communication constraints rather than algorithmic processing. This validates the system's capacity to adapt to dynamic user behavior via continuous learning with minimal operational impact.

6.5. Discussion of results

The results presented in this section reveal significant trends that warrant deeper analysis. In this section, we will discuss the significance of these findings and their implications within the context of the study. An important point to highlight is that, in addition to using three datasets in our experiments, we simulated multiple different actuator devices with which the agents worked. We used 3 control agents for actuator devices during experiments with the Real-world dataset, 13 with the AC dataset, and 17 with the BC dataset. In other words, the generalization capacity was tested and observed even more times.

From a general perspective, both RL algorithms demonstrated robust convergence, achieving HHS scores exceeding 96.88% in the simple-state approach and 93.80% in the composite-state approach, effectively

Table 8
Average experiment runtimes and latency metrics for training and inference of the RL agents (in seconds).

Dataset	Approach	Algorithm	Experiment time (s)	Train step latency (s)	Inference step latency (s)
AC dataset	simple	DQL	1258.11	0.0023	0.0029
		DSGnSARSA	1827.29	0.0015	0.0018
		SL baseline	996.78	0.0010	0.0020
	composite	DQL	1495.72	0.0018	0.0024
		DSGnSARSA	2086.03	0.0015	0.0022
		SL baseline	1210.02	0.0009	0.0020
BC dataset	simple	DQL	2334.93	0.0022	0.0024
		DSGnSARSA	3312.16	0.0015	0.0022
		SL baseline	1881.49	0.0010	0.0020
	composite	DQL	2811.93	0.0016	0.0029
		DSGnSARSA	3684.41	0.0014	0.0024
		SL baseline	2283.47	0.0010	0.0024
Real-world dataset	simple	DQL	58.04	0.0022	0.0015
		DSGnSARSA	81.95	0.0017	0.0017
		SL baseline	33.76	0.0009	0.0009
	composite	DQL	80.70	0.0022	0.0019
		DSGnSARSA	109.21	0.0015	0.0019
		SL baseline	48.01	0.0009	0.0010

satisfying user requirements in all scenarios. However, statistical analysis reveals distinct performance dynamics. In the synthetic environments (AC and BC datasets), the hypothesis tests revealed statistically significant differences among the methods, with DSGnSARSA presenting lower HHS values than DQL, and both reinforcement learning agents presenting lower HHS values than the SL baseline. In the real-world dataset, the statistical analysis indicated lower HHS values for DQL than for DSGnSARSA, while both RL agents achieved lower HHS values than the SL baseline.

Additionally, it was observed that the two reward-based action-selection strategies in our proposal performed better with the DQL algorithm, but only in experiments with synthetic datasets. For the DSGnSARSA algorithm, the reward-based functions performed better only in the simple-state approach with the Real-world dataset. Designed to handle both smart home usage history and real-time user interactions, our RS was evaluated using an offline evaluation protocol. The experiments leveraged a real-world dataset alongside two synthetic datasets to validate the system and assess the performance of the employed algorithms in both recorded real-world and simulated scenarios.

Furthermore, a more holistic view of the proposed system highlights that its impact goes beyond recommendation accuracy and algorithmic performance. By incorporating implicit feedback, usage history, and continuous user interaction, the system simultaneously contributes to resident comfort, energy efficiency, and adaptability to behavioral changes over time. The high HHS values indicate that the recommended actions align with user preferences, reducing manual interventions and improving the overall experience in the smart environment. At the same time, the model's ability to adapt daily suggests potential to optimize device usage, avoid unnecessary activations, and encourage more efficient energy consumption patterns. Moreover, evaluation across scenarios with multiple actuators, users, and routines demonstrates that the system is sensitive to contextual variations, adjusting its decisions as habits evolve or become more complex. This integrated perspective reinforces that the effectiveness of a RS in SHs should be assessed not only through accuracy metrics, but also through its ability to balance well-being, sustainability, and resilience to the dynamics of human behavior.

7. Conclusion

This work contributes to the state of the art by proposing a RS for SH that models implicit feedback and its corresponding reward function, enabling autonomous personalization on smart devices. In addition, it presents a comparison between on-policy and off-policy reinforcement learning algorithms applied to the SH domain. The research also proposes and analyzes two action-selection algorithms for reinforcement learning systems operating in environments with the potential for routine change, as well as a new metric for evaluating predictions in smart devices with composite states.

Throughout this work, we proposed an event-driven RS architecture focused on SHs that recommends actions for actuators in a smart environment. The unique features of this proposal include the approach to devices with simple and compound states, the modeling of implicit feedback, and the integration of user interaction into the RS learning process in dynamic environments. While there are studies proposing architectures for similar scenarios, mention of user feedback is rare. In our architecture, this feedback is used to improve the learning models, allowing them to approximate the resident's decisions as closely as possible and adapt to routine changes.

The conducted experiments evaluated the behavior of two RL algorithms, benchmarking them against an online SL baseline. The performance was analyzed daily to verify the model's ability to adapt to new resident routines, spanning diverse user patterns, counts, and smart device configurations. The proposed approach achieved HHS values exceeding 96.88% in simple-state scenarios and 93.80% in composite-state scenarios.

7.1. Limitations

It is essential to acknowledge the inherent limitations of the methodology and the research's scope, which may affect the generalizability of the results. The main limitations identified are:

- **Lack of real-time deployment:** The experiments performed in this work, including those using the real-world dataset, were conducted retrospectively (offline experiments). The proposed RS was not deployed in a live smart home environment with active users. Therefore, factors such as the user's psychological reaction to recommendations (e.g., trust, annoyance, or habit adaptation over time) could not be directly measured.
- **Limited variety of reinforcement learning algorithms explored:** This study focused on the application of only two RL algorithms. The vast range of existing algorithms in the RL field, including those with distinct approaches (such as actor-critic algorithms), was not explored. This restriction may mean that other RL algorithms could offer better results for the SHs scenario.
- **Sample Size (N = 17):** The sample size for the Real-world dataset appears very limited (N = 17), which raises concerns about generalizability. While the real-world dataset is necessarily small due to the constraints of collecting high-quality, longitudinal smart-home interaction data, it serves a complementary role in our evaluation. Nev-

ertheless, we agree that a small cohort may not adequately represent the heterogeneity of smart home users in terms of daily routines, device usage patterns, behavioral variability, and adaptation to recommendations over time. This limitation is particularly relevant for reinforcement learning–based systems, where policy convergence and reward dynamics can be sensitive to user-specific behaviors. For this reason, the Real-world dataset is interpreted primarily as a proof of applicability under realistic conditions, while broader generalization claims rely on complementary experiments conducted with larger synthetic datasets and multiple simulated actuator configurations.

7.2. Future works

Based on the challenges and lessons learned, we outline the following directions for future work:

- **Experimentation in real environments with users**: Although simulated environments provide high representativeness and experimental control, real-world smart home settings are inherently more complex, involving multiple rooms, multiple occupants with potentially conflicting routines, and dynamic environmental conditions. We therefore plan to extend the real-world deployment to larger and more heterogeneous households, explicitly modeling multi-user interactions, shared devices, and spatial context across rooms. In addition, future work will address operational constraints that are not fully captured in simulation, such as network latency, intermittent connectivity, sensor noise, and actuator malfunctions. These aspects will be studied through controlled fault-injection experiments and asynchronous communication models, enabling the evaluation of system robustness and adaptability under realistic failure conditions. This extended validation aims to narrow the gap between simulated and real-world environments and to further improve the ecological validity of the proposed approach.
- **Integrating explicit feedback into the system:** This type of feedback can require more effort from the user, which often discourages its use. However, the approach's viability increases if the option is offered, allowing engaged users to provide direct feedback whenever they wish.
- **Explaining the recommendations:** Recommendation systems in smart homes are relatively new, and users in this context often have limited prior experience interacting with such systems. As a result, users may wish to understand why specific recommendations are being made. Therefore, a promising direction for future work is to develop explainability features that allow users to view and interpret the rationale behind the recommendations.
- **Privacy and security:** There is extensive research on privacy and security in recommendation systems and IoT. Given that smart home environments handle sensitive and personal data, it is essential to go beyond system functionality and develop studies that measure and evaluate the privacy and security properties of these algorithms in real-world settings. Accordingly, a focused investigation of these aspects is an important direction for future work.
- **Experiment with other RL approaches:** Based on the results obtained in this paper, we envisage that additional RL methods can be applied in the smart home context, as well as that new RL methods can be developed specifically for smart home applications. Thus, as future work, we aim to develop new algorithms and propose adaptations for smart homes based on current RL approaches.
- **Qualitative analyses in a real-world environment:** Considering the future work outlined above, we plan to combine experimentation in real-world environments with privacy- and security-aware system design to develop a deployable solution capable of qualitatively evaluating user feedback and understanding how the proposed system improves users' daily quality of life. In addition, we intend to develop a feature that provides users with real-time interpretability of the generated recommendations, allowing them to better understand system behavior and decision-making processes.

The results demonstrate that this research contributed to the RSs and SHs scientific communities, advancing an underexplored topic that offers several opportunities for improvement.

CRediT authorship contribution statement

Denis Robson Dantas Boaventura: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization; **Eduardo Ferreira da Silva:** Writing – Original Draft, Writing – review & editing; **Mayki dos Santos Oliveira:** Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing; **Diego Corrêa da Silva:** Writing – original draft, Writing – review & editing.; **Frederico Araújo Durão:** Conceptualization, Writing – original draft, Writing – review & editing, Supervision, Project administration.

Data availability

The code and data used in this work are referenced via a GitHub URL in a footnote in the Introduction.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Afsar, M. M., Crump, T., & Far, B. (2022). Reinforcement learning based recommender systems: A survey. *ACM Computing Surveys*, *55*(7). https://doi.org/10.1145/3543846

Alomar, S., Alashwan, M., Busbaih, N., & Vizcarra, C. (2022). Smart home: Household appliance usage recommender and monitoring system (HARMS). In *2022 2nd International conference on computing and information technology (ICCIT)* (pp. 278–284). https://doi.org/10.1109/ICCIT52419.2022.9711598

Asare-Bediako, B., Kling, W. L., & Ribeiro, P. F. (2013). Multi-agent system architecture for smart home energy management and optimization. In *IEEE PES ISGT Europe 2013* (pp. 1–5). https://doi.org/10.1109/ISGTEurope.2013.6695331

Ashton, K. (1999). That 'internet of things' thing. *RFID Journal* . https://api.semanticscholar.org/CorpusID:109032799.

B, D., Mathala, N. K., A, S., & K, I. (2024). Secure and energy-efficient smart home automation: A user-based fingerprint security system. In *2024 Third international conference on intelligent techniques in control, optimization and signal processing (INCOS)* (pp. 1–6). https://doi.org/10.1109/INCOS59338.2024.10527495

Carvalho, D. S., Santos, P. A., & Melo, F. S. (2025). Reinforcement learning in convergently non-stationary environments: Feudal hierarchies and learned representations. *Artificial Intelligence*, *347*, 104382. https://doi.org/10.1016/j.artint.2025.104382

Chen, X., Yao, L., McAuley, J., Zhou, G., & Wang, X. (2023). Deep reinforcement learning in recommender systems: A survey and new perspectives. *Knowledge-Based Systems*, *264*, 110335. https://doi.org/10.1016/j.knosys.2023.110335

CHÉ, N. K., Pardons, N., Vanrompay, Y., Preuveneers, D., & Berbers, Y. (2010). An intelligent domotics system to automate user actions. In J. C. Augusto, J. M. Corchado, P. Novais, & C. Analide (Eds.), *Ambient intelligence and future trends-international symposium on ambient intelligence (ISAmi 2010)* (pp. 201–204). Berlin, Heidelberg: Springer Berlin Heidelberg.

Cook, D. J., Youngblood, M., Heierman, E. O., Gopalratnam, K., Rao, S., Litvin, A., & Khawaja, F. (2003). MavHome: An agent-based smart home. In *Proceedings of the first IEEE international conference on pervasive computing and communications, 2003. (percom 2003)*. (pp. 521–524). https://doi.org/10.1109/PERCOM.2003.1192783

da Silva, D. C., Robson Dantas Boaventura, D., dos Santos Oliveira, M., Pereira Santos Junior, J., Ferreira da Silva, E., de Almeida, E. S., Prazeres, C. V. S., do Carmo Machado, I., Leone Maciel Peixoto, M., Bittencourt Figueiredo, G., & Durão, F. A. (2025). Evaluating multi-label machine learning models for smart home environments. *Software: Practice and Experience*, *55*(9), 1427–1444. https://doi.org/10.1002/spe.3428

Davidoff, S., Lee, M. K., Yiu, C., Zimmerman, J., & Dey, A. K. (2006). Principles of smart home control. In *Proceedings of the 8th international conference on ubiquitous computing* UbiComp'06 (p. 19–34). Berlin, Heidelberg: Springer-Verlag. https://doi.org/10.1007/11853565_2

Ditzler, G., Roveri, M., Alippi, C., & Polikar, R. (2015). Learning in nonstationary environments: A survey. *IEEE Computational Intelligence Magazine*, *10*(4), 12–25. https://doi.org/10.1109/MCI.2015.2471196

dos Santos, O. M., Boaventura, D. R. D., Silva, E. F. D., Pires, J. M., Copque, I. S., Santos, B. P. d., Machado, I. d. C., Prazeres, C. V. S., Peixoto, M. L. M., Figueiredo, G. B., Almeida, E. S. d., Iwamoto, H. M., & Durão, F. A. (2025). Hestia: A home environment simulator targeting inhabitant activities. TechRxiv . https://doi.org/10.36227/techrxiv.173888093.30111102/v2

Gokul, V., Kannan, P., Kumar, S., & Jacob, S. G. (2016). Deep q-learning for home automation. *International Journal of Computer Applications*, *152*(6), 1–5. https://doi.org/10.5120/ijca2016911873

Himeur, Y., Alsalemi, A., Al-Kababji, A., Bensaali, F., Amira, A., Sardianos, C., Dimitrakopoulos, G., & Varlamis, I. (2021). A survey of recommender systems for energy efficiency in buildings: Principles, challenges and prospects. *Information Fusion*, *72*, 1–21. https://doi.org/10.1016/j.inffus.2021.02.002

Jeon, H., Kim, J., Yoon, H., Lee, J., & Kang, U. (2022). Accurate action recommendation for smart home via two-level encoders and commonsense knowledge. In *Proceedings of the 31st ACM international conference on information & knowledge management* CIKM '22 (p. 832–841). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3511808.3557226

Jiménez-Bravo, D. M., Pérez-Marcos, J., H. De la Iglesia, D., Villarrubia González, G., & De Paz, J. F. (2019). Multi-agent recommendation system for electrical energy optimization and cost saving in smart homes. *Energies*, *12*(7). https://doi.org/10.3390/en12071317

Kaur, E., Sharma, S., Verma, A., & Singh, M. (2019). An energy management and recommender system for lighting control in internet-of-energy enabled buildings. *IFAC-PapersOnLine*, *52*(4), 288–293. IFAC Workshop on Control of Smart Grid and Renewable Energy Systems CSGRES 2019. https://doi.org/10.1016/j.ifacol.2019.08.213

Kazemi, S. M., Goel, R., Eghbali, S., Ramanan, J., Sahota, J., Thakur, S., Wu, S., Smyth, C., Poupart, P., & Brubaker, M. A. (2019). Time2vec: Learning a vector representation of time. http://arxiv.org/abs/1907.05321.

Khezzane, K., Doumbia, M. L., Khoucha, F., & Hissem, S. (2024). Multi-layer multi-objective optimization for smart home energy management system with integration of user satisfaction concept. In *2024 Third international conference on sustainable mobility applications, renewables and technology (SMART)* (pp. 1–4). https://doi.org/10.1109/SMART63170.2024.10815417

Kim, M. J., Cho, M. E., & Jun, H. J. (2020). Developing design solutions for smart homes through user-centered scenarios. *Frontiers in Psychology*, *11*. https://doi.org/10.3389/fpsyg.2020.00335

Luo, F., Ranzi, G., Kong, W., Liang, G., & Dong, Z. Y. (2021). Personalized residential energy usage recommendation system based on load monitoring and collaborative filtering. *IEEE Transactions on Industrial Informatics*, *17*(2), 1253–1262. https://doi.org/10.1109/TII.2020.2983212

Mirra, J., Silva, F., & Analide, C. (2018). Reinforcement learning based approach for smart homes. In *Intelligent environments 2018* (pp. 38–47). (*vol. 23*). Ambient Intelligence and Smart Environments. https://doi.org/10.3233/978-1-61499-874-7-38

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing atari with deep reinforcement learning. https://arxiv.org/abs/1312.5602.

Morales, M. (2020). Grokking deep reinforcement learning. Manning.

Mozer, M. C. (1998). The neural network house: An environment that adapts to its inhabitants. In *Proc. AAAI spring symp. intelligent environments* (pp. 110–114). (*vol. 58*).

Padakandla, S., Prabuchandran, K. J., & Bhatnagar, S. (2020). Reinforcement learning algorithm for non-stationary environments. *Applied Intelligence*, *50*(11), 3590–3606. https://doi.org/10.1007/s10489-020-01758-5

Rashdan, M., Ahmad Almhaileej, H. Y., Almutairi, S. A., Eissa Ashkanani, F. K., Alhashimi, A. H., & Yousef Madoh, R. H. (2025). IoT-based home automation system for smart living. In *2025 6th International conference on bio-engineering for smart technologies (bioSMART)* (pp. 1–4). https://doi.org/10.1109/BioSMART66413.2025.11046048

Reaz, M. B. I., Assim, A., Ibrahimy, M., Choong, F., & Mohd-Yasin, F. (2008). Hardware simulation of home automation using pattern matching and reinforcement learning for disabled people. In *IC-AI* (pp. 213–218).

Ricci, F., Rokach, L., & Shapira, B. (2022). Recommender systems handbook. In F. Ricci, L. Rokach, & B. Shapira (Eds.), *Recommender systems handbook* (pp. 1–35). New York, NY: Springer US. https://doi.org/10.1007/978-1-0716-2197-4

Rodríguez Fernández, M., Cortés García, A., González Alonso, I., & Zalama Casanova, E. (2015). Using the big data generated by the smart home to improve energy efficiency management. *Energies*, (p. 249–260). https://doi.org/10.1007/s12053-015-9361-3

Sun, J., Shen, Y., Wang, Y., & Wang, Y. (2025). A memristor-based neural network circuit with retrospective revaluation effect and application in intelligent household robots. *IEEE Transactions on Neural Networks and Learning Systems*, *36*(6), 11466–11478. https://doi.org/10.1109/TNNLS.2025.3539842

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction. (2nd ed.). The MIT Press. http://incompleteideas.net/book/the-book-2nd.html.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Proceedings of the 31st international conference on neural information processing systems* NIPS'17 (p. 6000–6010). Red Hook, NY, USA: Curran Associates Inc.

Wang, H., Cao, H., & Yang, L. (2024). Machine learning-driven multidomain nanomaterial design: From bibliometric analysis to applications. *ACS Applied Nano Materials*, *7*(23), 26579–26600. https://doi.org/10.1021/acsanm.4c04940

Wang, H., Yang, L., Leng, D., Du, Y., & Ning, H. (2025). Accelerating the discovery and optimization of metal-organic framework materials via machine learning. *Advances in Colloid and Interface Science*, *346*, 103671. https://doi.org/10.1016/j.cis.2025.103671

Yang, L., Wang, H., Leng, D., Fang, S., Yang, Y., & Du, Y. (2024). Machine learning applications in nanomaterials: Recent advances and future perspectives. *Chemical Engineering Journal*, *500*, 156687. https://doi.org/10.1016/j.cej.2024.156687

Yang, Y., Wu, L., Yin, G., Li, L., & Zhao, H. (2017). A survey on security and privacy issues in internet-of-things. *IEEE Internet of Things Journal*, *4*(5), 1250–1258. https://doi.org/10.1109/JIOT.2017.2694844

Yu, L., Xie, W., Xie, D., Zou, Y., Zhang, D., Sun, Z., Zhang, L., Zhang, Y., & Jiang, T. (2020). Deep reinforcement learning for smart home energy management. *IEEE Internet of Things Journal*, *7*(4), 2751–2762. https://doi.org/10.1109/JIOT.2019.2957289

Zhang, S., Tay, Y., Yao, L., Sun, A., & Zhang, C. (2022). Deep learning for recommender systems. In F. Ricci, L. Rokach, & B. Shapira (Eds.), *Recommender systems handbook (3rd edition)* (pp. 173–210). Springer. https://doi.org/10.1007/978-1-0716-2197-4_5